

Con toda intención ofrecemos una exposición detallada: se busca ejemplificar de modo exhaustivo que no hay decisiones "mágicas" ni injustificables en el diseño orientado a objetos. Podemos explicar y aprender racionalmente otras asignaciones de responsabilidades y la elección de interacciones de objetos.

Las siguientes indicaciones le servirán para repasar los pasos de la preparación de un diagrama de interacción.

Para elaborar diagramas de interacción:

1. Prepare un diagrama individual para cada operación del sistema en el paso iterativo actual.
 - Por cada evento del sistema construya un diagrama que lo incluya como mensaje inicial.
2. Si el diagrama se vuelve complejo, divídalo en otros más pequeños.
3. Con las responsabilidades contractuales, las poscondiciones y la descripción del caso de uso como punto de partida, diseñe un sistema de objetos interactuantes para que realicen las tareas. Aplique los patrones GRASP y otros para elaborar un buen diseño.

19.2 Diagramas de interacción y otros artefactos

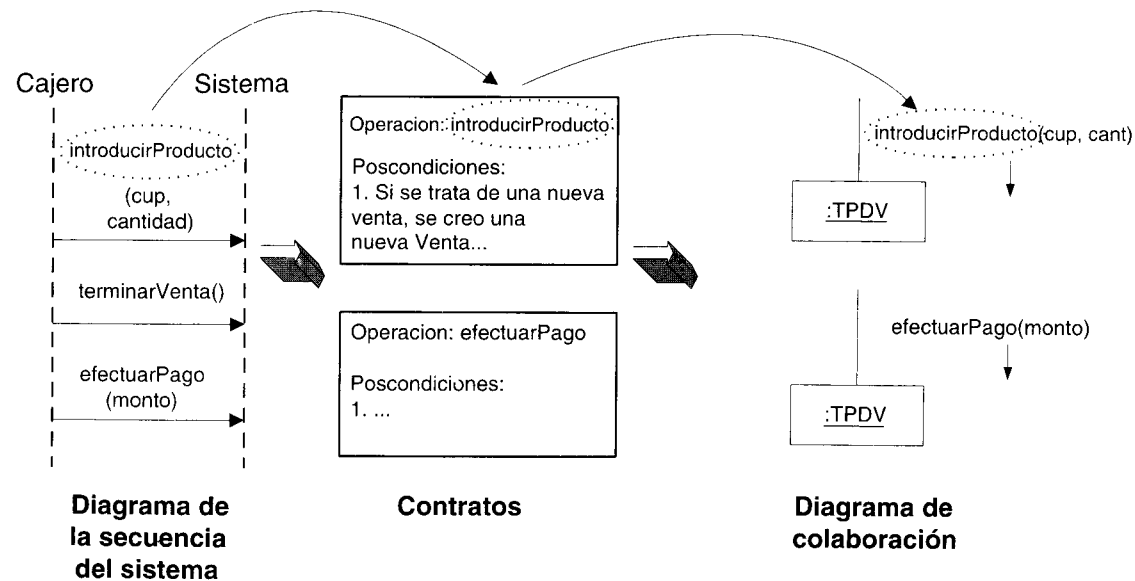


Figura 19.1 Relación entre artefactos desde la perspectiva de los eventos de un sistema.

La figura 19.1 muestra visualmente la relación entre los artefactos.

- Los casos de uso indican los eventos del sistema que se muestran explícitamente en los diagramas de la secuencia de él.
- La mejor conjetura inicial sobre el efecto producido por los eventos del sistema se describe en los contratos de sus operaciones.
- Los eventos del sistema representan mensajes iniciadores de los diagramas de interacción que describen visualmente cómo los objetos interactúan para realizar las tareas en cuestión.
- Los diagramas de interacción incluyen la interacción de los mensajes entre los objetos que se definen en el modelo conceptual y otras clases de objetos.

19.2.1 Los diagramas de interacción y los eventos del sistema

En la iteración actual de la aplicación del punto de venta estamos considerando dos casos de uso y sus eventos sistemáticos asociados:

- Comprar Productos
 - introducirProducto
 - terminarVenta
 - efectuarPago
- Inicio
 - inicio

Por cada evento del sistema construya un diagrama de colaboración cuyo mensaje inicial sea el de sus eventos.

Habrà, pues, cuatro diagramas de interacción por lo menos: uno por cada evento del sistema. Con el patrón Controlador, la clase *TPDV* puede escogerse como controlador para que maneje los eventos (figura 19.2).

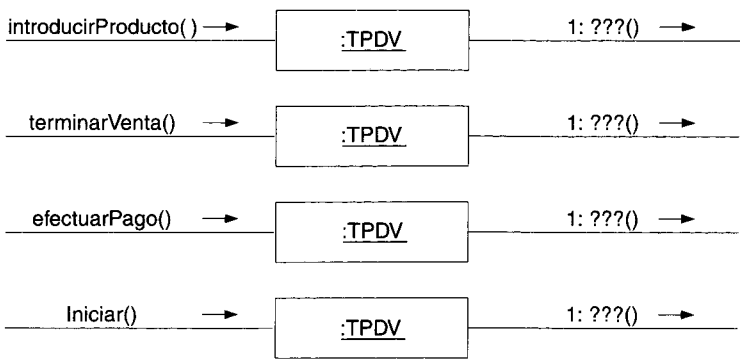


Figura 19.2 Eventos del sistema.

19.2.2 Los diagramas de interacción y los contratos

Con las responsabilidades y las poscondiciones del contrato, así como también los casos de uso como punto de partida, diseñe un sistema de objetos interactuantes para que realicen las tareas.

Por cada operación del sistema existe un contrato que estipula lo que deben lograr las operaciones. Por ejemplo:

Contrato	
Nombre:	introducirProducto (cup : número cantidad : entero).
Responsabilidades:	Capturar (registrar) la venta de un producto y agregarla a la venta. Desplegar la descripción del producto y su precio.
Poscondiciones:	

- Si se trata de una nueva venta, se creó una *Venta* (creación de instancia).
- Si se trata de una nueva venta, se asoció la nueva *Venta* a *TPDV*(asociación formada).
- y así sucesivamente...

En cada contrato revisamos los cambios de estado de las responsabilidades y las poscondiciones, diseñando al mismo tiempo interacciones de mensajes —incluidos en el diagrama de colaboración— para atender los requerimientos. Nótese que, en caso de omitir la preparación del contrato, de todos modos deberíamos elaborar los diagramas de interacción retornando a los casos de uso y reflexionando sobre lo que debemos lograr. No obstante, los contratos organizan y aíslan la información en un formato funcional, al mismo tiempo que estimulan la investigación en la fase de análisis y no en la de diseño.

Por ejemplo, en esta operación parcial del sistema *introducirProducto*, en la figura 19.3 incluimos un diagrama parcial de colaboración que satisface el cambio de estado de la creación de *Venta*.

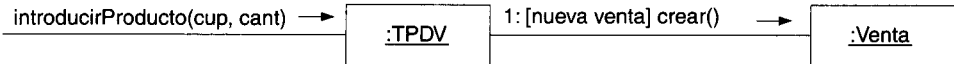


Figura 19.3 Diagrama parcial de colaboración.

19.2.3 Las poscondiciones no son más que una estimación

Los diagramas pueden prepararse con el propósito de cumplir las poscondiciones del contrato. Pero es indispensable reconocer que las poscondiciones definidas de antemano no son sino una excelente conjetura o estimación inicial de lo que se pretende alcanzar. Tal vez no sean muy exactas. Lo mismo sucede con el modelo conceptual: es un punto de partida que contendrá errores y omisiones. En los contratos debemos ver un mero punto de partida para establecer lo que se hará, pero sin sentirnos obligados por ellos. Lo más probable es que algunas poscondiciones no sean necesarias y que haya algunas tareas por realizar que aún no hemos descubierto. Una ventaja del desarrollo iterativo radica en que brinda un soporte espontáneo a la detección de nuevos resultados del análisis y del diseño durante las fases de solución y de construcción.

El desarrollo iterativo se propone alcanzar un nivel “razonable” de información en la fase de análisis, llenando después los detalles en la del diseño. De modo análogo, busca también lograr un nivel “razonable” de resultados del diseño durante la fase de diseño, llenando después los detalles en la de implementación (codificación). La definición de “razonable” es, naturalmente, cuestión de opinión; en este libro procuramos presentar un grado razonable de actividad en cada paso.

19.2.4 Los diagramas de colaboración y el modelo conceptual

Algunos de los objetos que interactúan a través de mensajes en los diagramas de colaboración provienen del modelo conceptual. En parte, la elección de la asignación acertada de las responsabilidades mediante los patrones GRASP se funda en la información contenida en el modelo conceptual. Según mencionamos en páginas anteriores, el actual modelo conceptual difícilmente será perfecto: se prevén errores y omisiones. Usted descubrirá conceptos que antes no estaban incluidos, omitirá los que ya habían sido identificados y hará lo mismo con las asociaciones y con los atributos.

19.3 Modelo conceptual del punto de venta

A manera de recordatorio incluimos la figura 19.4, donde el lector podrá examinar el modelo conceptual en la aplicación al punto de venta. Se trata de objetos que pueden participar en las interacciones referentes a ellos. ¿Acaso los únicos modelos de los objetos en interacción son los que figuran en este modelo? De ninguna manera; en la fase de diseño conviene descubrir otros tipos que no se incluyeron en los análisis anteriores. Y cuando se descubran los incorporaremos al modelo.

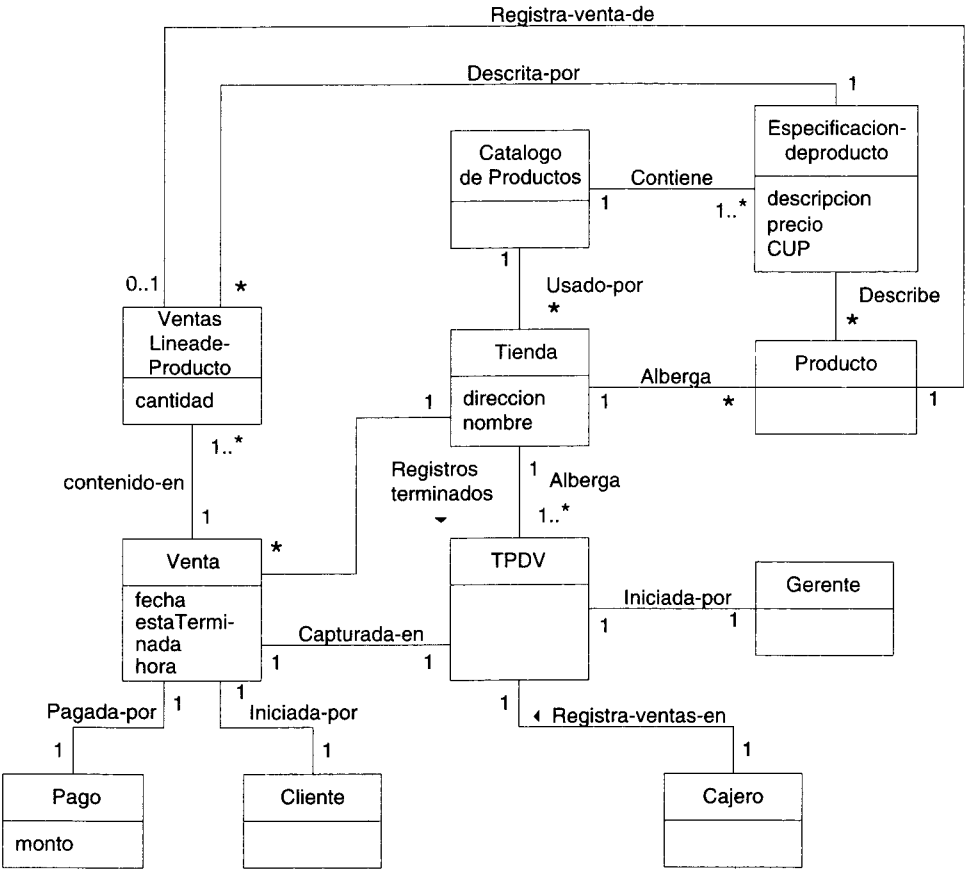


Figura 19.4 Modelo conceptual del punto de venta.

19.4 Diagramas de colaboración para la aplicación TPDV

En esta sección estudiaremos las decisiones y selecciones hechas al preparar un diagrama de colaboración a partir de los patrones GRASP. Nuestras explicaciones son deliberadamente pormenorizadas, pues queremos demostrar que no se da una adhesión irreflexiva cuando se crean diagramas de colaboración bien diseñados: su construcción se basa más bien en principios justificables.

19.5 El diagrama de colaboración: introducirProducto

La operación del sistema *introducirProducto* ocurre cuando un cajero captura el CUP (código universal de producto) y la cantidad del objeto a comprar. A continuación se incluye íntegramente un contrato; recuerde el lector que se trata de una estimación inicial y quizá no sea completa:

Contrato	
Nombre:	introducirProducto (cup : número cantidad: entero).
Responsabilidades:	Capturar (registrar) la venta de un producto y agregarla a la venta. Desplegar la descripción del producto y su precio.
Tipo:	Sistema.
Referencias cruzadas:	Funciones del sistema: R1.1, R1.3, R1.9. Casos de uso: Comprar productos.
Notas:	Usar el acceso superrápido a la base de datos.
Excepciones:	Si el CUP no es válido, indicar que se cometió un error.
Salida:	
Precondiciones:	El sistema conoce el CUP.
Poscondiciones:	

- Si se trata de una nueva venta, se crea una *Venta* (creación de instancia)
- Si se trata de una nueva venta, se asocia la nueva *Venta* a la instancia *TPDV* (formación de asociación)
- Se creó una instancia *VentasLineadeProducto* (creación de instancia)
- Se asoció la instancia *VentasLineadeProducto* a la instancia *Venta* (formación de asociación)
- Se asignó a *VentasLineadeProducto.cantidad* el valor en *cantidad* (modificación de atributo)
- Se asoció la instancia *VentasLineadeProducto* a *EspecificaciondeProducto* basado esto en la correspondencia del CUP (formación de asociación)

Se construirá un diagrama de colaboración que cumple con las poscondiciones de *introducirProducto*. Como se señaló en el capítulo dedicado a los patrones GRASP, en cada elección de un mensaje se requiere asignar una responsabilidad; los patrones GRASP servirán para escoger y justificar los mensajes que se envíen a una clase de objetos.

19.5.1 Selección de la clase Controlador

Nuestra primera decisión se referirá a la elección del controlador que se encargue del mensaje de las operaciones del sistema *introducirProducto*. Desde el punto de vista de este patrón, disponemos de las siguientes opciones:

representar todo el sistema (controlador de fachada)	<i>TPDV</i> , o una clase nueva como <i>SistemaAlMenudeo</i>
representar la empresa o la organización total (controlador de fachada)	<i>Tienda</i>
representar algo del mundo real que sea activo (por ejemplo, el papel de una persona) y que pueda participar en la tarea (controlador de papeles)	<i>Cajero</i>
representa un manejador artificial de todas las operaciones del sistema en un caso de uso (controlador de casos de uso).	<i>ManejadordeComprar-Productos</i>

Es satisfactorio elegir un controlador de fachada como *TPDV*, si el sistema ofrece pocas operaciones y si ese patrón no asume demasiadas responsabilidades (en otras palabras, si empieza a perder cohesión). Escoger un controlador de papeles o de casos de uso es una decisión acertada, cuando el sistema tiene demasiadas operaciones y queremos distribuir las responsabilidades para que las clases controlador no se atiborren ni pierdan su orientación central (o sea, que sean cohesivas). En este caso, *TPDV* será suficiente porque el sistema tiene pocas operaciones.

Es importante darse cuenta que se trata de una instancia de *TPDV* de un objeto en "territorio de software". No es una terminal real de punto de venta, sino una abstracción de software que representa el registro.

Así pues, el diagrama de colaboración de la figura 19.5 comienza enviando el mensaje *introducirProducto* a una instancia de *TPDV* mediante un CUP y un parámetro de cantidad.

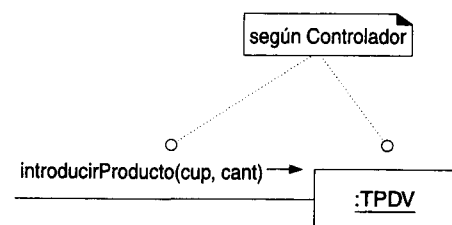


Figura 19.5 Aplicación del patrón Controlador de GRASP.

19.5.2 Presentación visual de la descripción y del precio

En virtud de un principio del diseño denominado **separación de modelo-vista**, no compete a los objetos del dominio (como *TPDV* o *Venta*) comunicarse con la capa de interfaz para el usuario (las ventanas gráficas, por ejemplo). De ahí que por ahora no nos ocupemos de la descripción del producto ni de su precio.

Lo único que se requiere en lo tocante a las responsabilidades del despliegue de la información es conocer los datos, como se verá en este caso.

19.5.3 Realización de una nueva venta

Es necesario examinar los requerimientos, expresados en los contratos, que han de atenderse en el software para lograr una ejecución correcta del sistema. Dos de las poscondiciones contractuales establecen lo siguiente:

- Si se trata de una nueva venta, se creó una *Venta* (creación de instancia).
- Si se trata de una nueva venta, se asoció la nueva *Venta* a *TPDV* (asociación formada).

Lo anterior significa una responsabilidad en la creación, y el patrón Creador de GRASP indica la asignación de la responsabilidad de generar una clase que agregue, contenga o registre la que va a ser creada.

Al analizar el modelo conceptual, o sea al reflexionar sobre los objetos del dominio, se descubre que en una instancia *TPDV* podemos ver el registro de una *Venta*,¹ por tanto, *TPDV* es un buen candidato para crear una *Venta*. Y al hacer que la cree, con el tiempo podremos asociar las dos, de modo que en operaciones futuras *TPDV* será una referencia a la instancia actual *Venta*.

Además de lo que acabamos de decir, cuando se crea la *Venta*, hay que generar una colección vacía (un contenedor como un *Vector* de Java) para que registre todas las instancias futuras *VentasLineadeProducto* que vayamos agregando. La instancia *Venta* le dará mantenimiento a esta colección, lo cual implica que según el patrón Creador la *Venta* es idónea para crearla.

En conclusión, la instancia *TPDV* crea la *Venta* y ésta, a su vez, produce una colección vacía, que está representada por un multiobjeto en el diagrama de colaboración.

¹ Nótese que *TPDV* es más o menos sinónimo de *registro*: un registro de ventas.

Entonces, el diagrama de colaboración de la figura 19.6 ilustra el diseño.

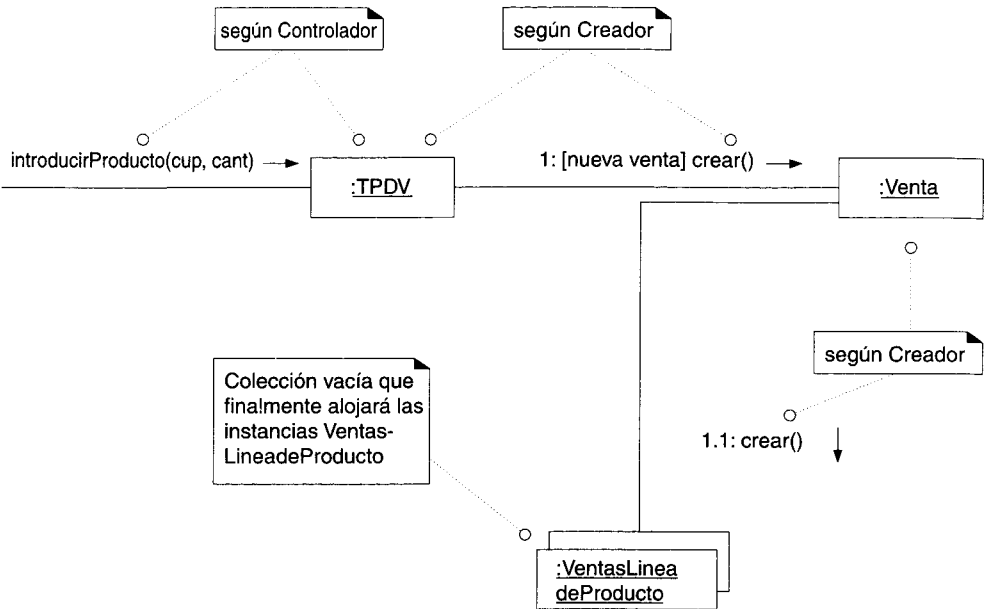


Figura 19.6 Creación de ventas.

19.5.4 Creación de una nueva instancia VentasLineadeProducto

Algunas otras poscondiciones contractuales de *introducirProducto* establecen lo siguiente:

- Se creó una instancia *VentasLineadeProducto* (creación de instancia).
- Se asoció *VentasLineadeProducto* a la *Venta* (asociación formada).
- Se asignó el valor a *VentasLineadeProducto.cantidad* de *cantidad* (modificación de atributo).
- Se asoció la instancia *VentasLineadeProducto* con una *EspecificaciondeProducto* a partir de la correspondencia en el CUP (asociación formada).

Lo anterior indica la responsabilidad de crear una instancia *VentasLineadeProducto*. El análisis del diagrama de clases revela que una *Venta* contiene los objetos *VentasLineadeProducto*; por tanto, el patrón Creador es un buen candidato para generar la línea de producto. Y al hacer que la *Venta* cree *VentasLineadeProducto*, podemos asociar la *Venta* con ésta a lo largo del tiempo al guardar la nueva instancia en su colección de línea de producto. Las poscondiciones indican que la nueva instancia *VentasLineadeProducto* necesita una cantidad al ser creada; por eso, la instancia *TPDV* ha de transmitirla a la *Venta*, que a su vez la transfiera como parámetro en el mensaje *crear*.

Así pues, basados en el patrón Creador a la *Venta* se le envía un mensaje *hacerLineadeProducto* para que produzca una instancia *VentasLineadeProducto*. La *Venta* crea una *VentasLineadeProducto* y luego almacena la nueva instancia en su colección permanente.

Los parámetros del mensaje *hacerLineadeProducto* son la *cantidad*, para que *VentasLineadeProducto* pueda registrarlo, y también la *EspecificaciondeProducto* que concuerde con el CUP.

19.5.5 Obtención de una instancia EspecificaciondeProducto

Necesitamos asociar la instancia *VentasLineadeProducto* a *EspecificaciondeProducto* que concuerde con el CUP que se recibe. Ello significa que se requiere recuperar una *EspecificaciondeProducto* basada en la correspondencia del CUP.

Antes de ver cómo lograr la consulta, es muy importante determinar quién se encargará de ella. Un primer paso de gran utilidad es el siguiente:

Comience asignando las responsabilidades definiéndolas con claridad.

Replanteamos el problema en los siguientes términos:

¿Quién debería asumir la responsabilidad de conocer una instancia *EspecificaciondeProducto* basado en que coincida el valor del CUP?

No estamos ante un problema de creación ni ante el de escoger un controlador. En la generalidad de los casos, el patrón Experto de GRASP es el principio que ha de aplicarse en primer lugar. Este patrón indica que el objeto con la información necesaria para desempeñar la responsabilidad debería encargarse de ella. ¿Quién está enterado de todo lo concerniente a *EspecificaciondeProducto*? El análisis del diagrama de clases revela que *CatalogodeProductos* contiene lógicamente todas las *EspecificacionesdeProductos* y así, de acuerdo con el patrón Experto, *CatalogodeProductos* es idóneo para asumir esta responsabilidad de consulta.

19.5.6 Visibilidad ante un CatalogodeProductos

¿Quién debería enviar el mensaje *especificación* al *CatalogodeProductos* para solicitar una *EspecificaciondeProducto*?

Es razonable suponer que una instancia *TPDV* y una de *CatalogodeProductos* fueron creadas durante el caso inicial de uso *Inicio* y que existe una conexión permanente entre el objeto *TPDV* y el objeto *CatalogodeProductos*. Esta suposición nos permitirá concluir que *TPDV* puede enviar el mensaje *especificación* a *Catálogo de productos*.

Ello significa la existencia de otro concepto en el diseño orientado a objetos: el de visibilidad. La **visibilidad** es la capacidad de un objeto para "ver" o tener una referencia a otro.

Para que un objeto envíe a otro un mensaje, debe ser visible al segundo.

Supondremos que *TPDV* tiene una conexión o referencia permanente a *CatalogodeProductos*; por tanto, es visible para esta instancia y de ahí que pueda enviarle mensajes como *especificación*.

En el siguiente capítulo trataremos más a fondo el tema de la visibilidad.

19.5.7 Recuperación de EspecificaciondeProducto a partir de una base de datos

En la versión final de una aplicación real al punto de venta, difícilmente todas las *EspecificacionesdeProducto* estarán en la memoria. Lo más probable es que se encuentren almacenadas en una base de datos relacional o de objetos y que sean recuperadas previa solicitud. No obstante, para no hacer compleja la exposición pospondremos el estudio de los problemas concernientes a la recuperación partiendo de una base de datos. Supondremos que todas las *EspecificacionesdeProducto* se hallan en la memoria. En el capítulo 38 abordaremos el tema del acceso a la base de datos de objetos persistentes.

19.5.8 El diagrama de colaboración introducirProducto

Una vez explicado quién debería crear la instancia *VentasLineadeProducto*, recuperar una *EspecificaciondeProducto* y realizar otras tareas, el diagrama de colaboración de la figura 19.7 refleja las decisiones sobre la asignación de responsabilidades y sobre la manera en que los objetos deberían interactuar. Observe que se reflexionó detenidamente para llegar a él, con base en los patrones GRASP; el diseño de las interacciones de los objetos y la asignación de responsabilidades requieren una seria deliberación.

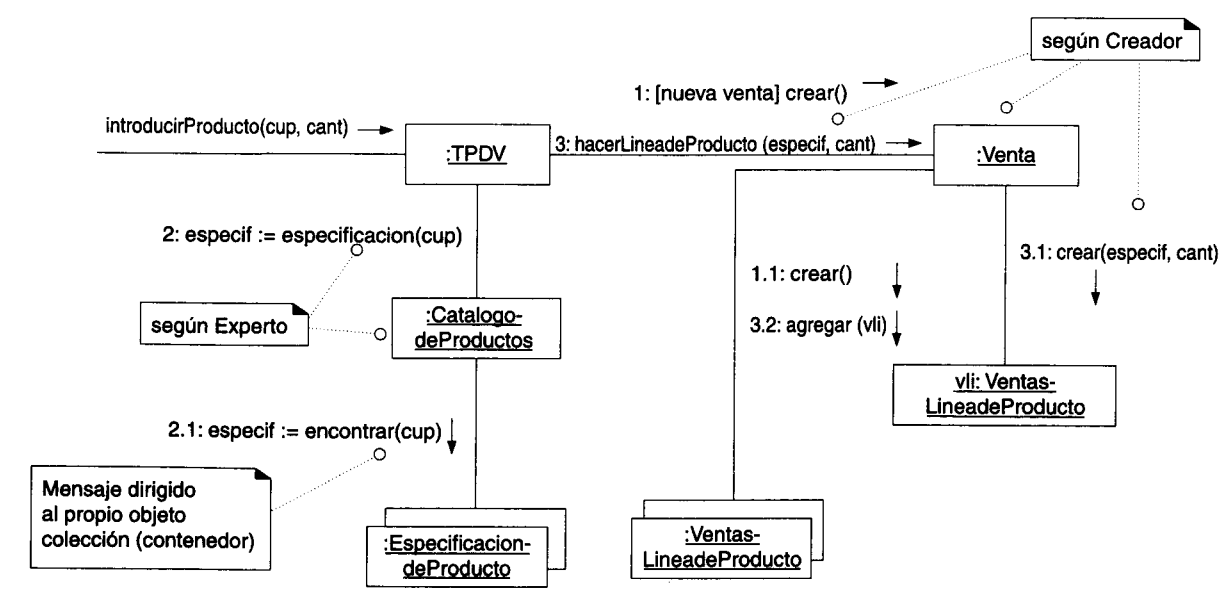


Figura 19.7 El diagrama de colaboración introducirProducto.

19.5.9 Mensajes a multiobjetos

Adviértase lo siguiente: la interpretación predeterminada de un mensaje enviado a un multiobjeto es que se envía implícitamente a todos los elementos de la colección/contenedor, pero también puede interpretarse como un mensaje dirigido al propio objeto colección. Ello ocurre especialmente en las operaciones genéricas de colección como *encontrar* y *agregar*. Por ejemplo, en el diagrama de colaboración *introducirProducto*:

- El mensaje *encontrar* (2.1) enviado al multiobjeto *EspecificaciondeProducto* se dirige una vez a la estructura de datos colección representada por el multiobjeto (un *Vector* en Java).
- El mensaje *crear* (1.1) enviado al multiobjeto *VentasLineadeProducto* tiene por objeto generar la estructura de datos colección representada por el multiobjeto (un *Vector* en Java, por ejemplo); su finalidad no es crear una instancia de la clase *VentasLineadeProducto*.
- El mensaje *agregar* (3.2) enviado al multiobjeto *VentasLineadeProducto* tiene por objeto incorporar un elemento a la estructura de datos colección representada por el multiobjeto.

19.6 El diagrama de colaboración: terminarVenta

La operación sistémica *terminarVenta* ocurre cuando un cajero oprime un botón para indicar la conclusión de una venta. He aquí el contrato íntegro:

Contrato	
Nombre:	terminarVenta().
Responsabilidades:	Registrar que se terminó de capturar los artículos de la venta y presentar visualmente el total de la venta.
Tipo:	Sistema.
Referencias cruzadas:	Funciones del sistema: R1.2. Casos de uso: Comprar Productos.
Notas:	
Excepciones:	Si está efectuándose una venta, indicar que se cometió un error.
Salida:	
Precondiciones:	El sistema conoce el CUP.
Poscondiciones:	
■ Se asignó a <i>Venta.estaTerminada</i> el <i>valor verdadero</i> (modificación de atributo).	

Se elaborará un diagrama de colaboración para cumplir las poscondiciones de *terminarVenta*. Como señalamos en el capítulo dedicado a los patrones GRASP, cada decisión sobre mensajes implica asignar una responsabilidad, y los patrones GRASP se emplearán para escogerlos y justificarlos.

19.6.1 Elección de la clase Controlador

Nuestra primera decisión se refiere a la asignación de la responsabilidad del mensaje *terminarVenta* de la operación del sistema. Basándonos en el patrón Controlador de GRASP, como lo hicimos con *introducirProducto*, seguiremos utilizando *TPDV* como controlador.

19.6.2 Establecimiento del atributo *Venta.estaTerminada*

Las poscondiciones contractuales establecen lo siguiente:

- Se ha establecido *Venta.estaTerminada* en *verdadero* (modificación de atributo).

Como siempre, el patrón Experto debería ser el primero a considerar, salvo que se tratara de un problema de controlador o de creación (pero no es así).

¿Quién debería encargarse de asignar el valor verdadero al atributo *estaTerminada* de la *Venta*?

Basándonos en el patrón Experto, debería ser la propia *Venta*, porque posee el atributo *estaTerminada* y le da mantenimiento. Así, *TPDV* enviará a *Venta* el mensaje *seTermina* para asignarle el valor *verdadero*. Obsérvese en la figura 19.8 el uso de una nota restrictiva que contiene pseudocódigo para aclarar el propósito de la operación *darPorTerminada*; esta última se recomienda cuando es necesario explicar los detalles de una operación.

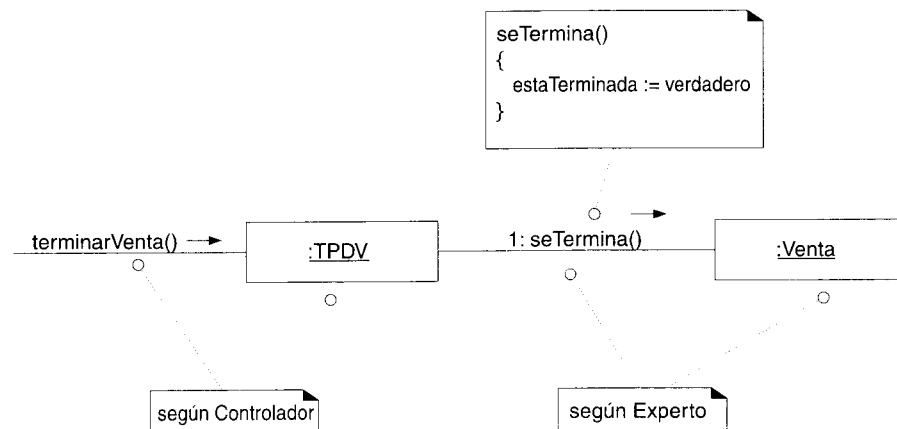


Figura 19.8 Terminación de la captura de un producto.

19.6.3 La presentación visual de información

En las responsabilidades del contrato *terminarVenta* se estipula que el total de la venta deberá mostrarse. Aclaremos la cuestión de quién debería encargarse de presentar visualmente la información, en una ventana gráfica por ejemplo. En síntesis, la capa de

los objetos del dominio —como *Venta*, *Tienda*, etc.— *no* debiera conocer las ventanas o la capa de presentación ni tener comunicación con ellas. Esto constituye la esencia del patrón *Separación Modelo-Vista*, que se explica en el capítulo 22.

Durante la preparación de los diagramas de colaboración, no se preocupe por mostrar la información salvo en la medida en que se conoce la que se necesita.

Por tanto, prescinda de los requerimientos que requieren presentar información, con excepción de lo siguiente:

Asegúrese de que toda la información que ha de mostrarse visualmente se conozca y pueda obtenerse de los objetos del dominio.

Por ejemplo, el requerimiento anterior indica que algún objeto debe conocer el total de la venta.

19.6.4 Cálculo del total de la venta

Gracias a que contamos con el patrón Separación Modelo-Vista, no es necesario preocuparnos de cómo mostrar el total de la venta, pero debemos cerciorarnos de conocer el total. Obsérvese que ninguna clase conoce en este momento el total; por ello, con un diagrama de colaboración debemos preparar un diseño de las interacciones de los objetos que atienden este requerimiento.

Como siempre, el patrón Experto habrá de ser el primero que consideremos a menos que nos enfrentemos a un problema de controlador o de creación (pero no es así).

Probablemente nos demos cuenta de que la *Venta* debería ser la encargada de conocer su total, pero examinemos el siguiente análisis para hacer muy claro —mediante un ejemplo— el proceso de razonamiento con que se obtiene un Experto.

1. Establezca la responsabilidad:
 - ¿Quién debería asumir la responsabilidad de conocer el total de la venta?
2. Sintetice la información requerida:
 - El total de la venta es la suma de los subtotales de todas las líneas de producto de la venta.
 - Subtotal de la línea de producto de la venta := cantidad de la línea de producto * precio del producto.
3. Incluya la información necesaria para desempeñar esta responsabilidad y las clases que conocen esta información.

Información necesaria para el total de la venta	Clase Experto
EspecificaciondeProducto.precio	EspecificaciondeProducto
VentasLineadeProducto.cantidad	VentasLineadeProducto
todas las VentasLineadeProducto en la Venta actual	Venta

Damos en seguida un análisis pormenorizado:

- ¿Quién debería asumir la responsabilidad de calcular el total de la *Venta*? Según el patrón Experto, tendría que ser la propia *Venta*, porque conoce todo lo concerniente a las instancias *VentasLineadeProducto* cuyos subtotales tienen que sumarse para calcular el total de la venta. Por consiguiente, *Venta* deberá tener la responsabilidad de conocer su total, que se implementó como una operación *total*.
- Para que una *Venta* calcule su total necesita el subtotal de cada instancia *VentasLineadeProducto*. ¿Quién debería asumir la responsabilidad de calcular el subtotal de esa instancia? Según el patrón Experto, debería ser la propia *VentasLineadeProducto*, porque conoce la cantidad y porque *EspecificaciondeProducto* está asociada a ella. Por tanto, *VentasLineadeProducto* tendrá la responsabilidad de conocer su subtotal, que se implementó como una operación *subtotal*.
- Para que *VentasLineadeProducto* calcule su subtotal, necesita el precio de *EspecificaciondeProducto*. ¿Quién debería asumir la responsabilidad de ofrecer este precio? De acuerdo con el patrón Experto, debería ser la propia *EspecificaciondeProducto*, porque encapsula el precio como atributo. Por tanto, *EspecificaciondeProducto* tendrá la responsabilidad de conocer su precio, que se implementó como una operación *precio*.

Aunque el análisis anterior es trivial en este caso y aunque no hace falta un grado de elaboración tan complicada, la misma estrategia con que se localiza un patrón Experto puede y debiera aplicarse en situaciones más difíciles. El lector se dará cuenta de que, una vez aprendidos los patrones GRASP, podrá realizar rápidamente este tipo de razonamiento en forma mental.

19.6.5 El diagrama de colaboración total de Venta

Una vez explicado lo anterior, conviene construir un diagrama de colaboración que indique lo que sucede cuando a una *Venta* se le envía un mensaje de *total*. El primer mensaje del diagrama es *total*, pero observe que no es un evento del sistema. De ello extraemos la siguiente observación:

No todo diagrama de colaboración comienza con un mensaje de eventos del sistema; puede empezar con cualquier mensaje cuyas interacciones desee mostrar el diseñador.

El diagrama de colaboración requerida se incluye en la figura 19.9. Primero, el mensaje de *total* se envía a la instancia *Venta*. Ésta enviará entonces un mensaje de *subtotal* a las instancias relacionadas de *VentasLineadeProducto*. Y a su vez esta instancia enviará un mensaje de *precio* a su correspondiente *EspecificacionesdeProducto*.

Dado que las operaciones aritméticas (por lo regular) no se explican gráficamente a través de mensajes, podemos describir los detalles de los cálculos incorporando al diagrama restricciones que definan los cálculos.

¿Quién enviará a la *Venta* el mensaje de *total*? Lo más probable es que lo haga un objeto de la capa de presentación; por ejemplo un Applet de Java.

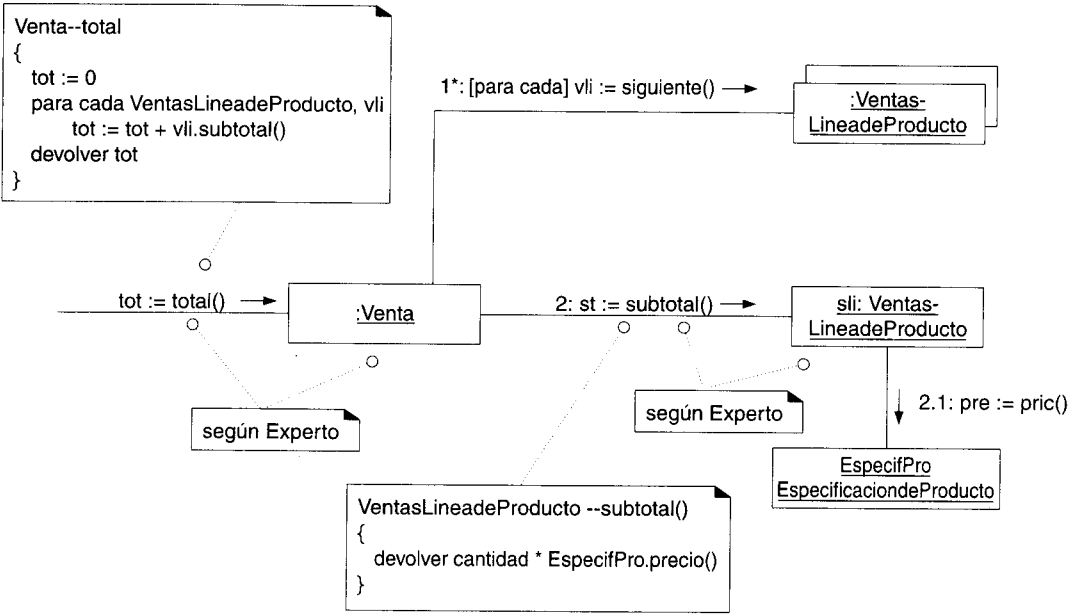


Figura 19.9 El diagrama de colaboración venta-total.

19.7 El diagrama de colaboración: efectuarPago

La operación sistémica *efectuarPago* ocurre cuando un cajero captura el efectivo ofrecido como pago. He aquí íntegramente el contrato:

Contrato

Nombre:	efectuarPago (monto: Número o Cantidad).
Responsabilidades:	Registrar el pago, calcular el saldo e imprimir el recibo.
Tipo:	Sistema.
Referencias cruzadas:	Funciones del sistema: R2.1. Casos de uso: Comprar Productos.
Notas:	
Excepciones:	Si la venta no está terminada, indicar que se cometió un error. Si el monto es menor que el total de la venta, indicar que se cometió un error.
Salida:	
Precondiciones:	
Poscondiciones:	
■ Se creó un <i>Pago</i> (creación de instancia). ■ Se asignó a <i>Pago.montoOfrecido</i> el valor del <i>monto</i> (modificación de atributo). ■ Se asoció <i>Pago</i> a <i>Venta</i> (relación formada). ■ Se asoció la <i>Venta</i> a <i>Tienda</i> para agregarla al registro histórico de ventas terminadas (relación formada).	

Se construirá un diagrama de colaboración para cumplir con las poscondiciones de *efectuarPago*. Como se señaló en el capítulo dedicado a los patrones GRASP, una decisión sobre los mensajes implica asignar una responsabilidad, y esos patrones servirán para escogerlos y justificarlos.

19.7.1 Elección de la clase Controlador

Nuestra primera decisión se refiere al manejo de la responsabilidad del mensaje de operación del sistema *efectuarPago*. Con base en el patrón Controlador de GRASP, igual que en el caso de *introducirProducto*, seguiremos empleando *TPDV* como Controlador. Es frecuente utilizar el mismo controlador a lo largo de un caso de uso.

19.7.2 Creación de Pago

Una de las poscondiciones contractuales estipula lo siguiente.

- Se creó un *Pago* (creación de instancia).

Se trata de una responsabilidad relativa a la creación y, por tanto, debería aplicarse el patrón Creador de GRASP.

¿Quién registra, agrega, utiliza más ampliamente o contiene un *Pago*? Como resulta atractivo afirmar que la instancia *TPDV* registra lógicamente un *Pago*, se pensará que es idóneo para ser usado. Además, cabe suponer que una *Venta* utilizará estrechamente un *Pago*; de ahí la posibilidad de que sea un buen candidato.

Otra manera de encontrar un creador consiste en servirse del patrón Experto para determinar quién es el Experto en los datos de inicialización: en este caso, el monto ofrecido. *TPDV* es el controlador que recibe el mensaje *efectuarPago(montoOfrecido)* de la operación del sistema; por eso al principio incluirá la cantidad ofrecida. En consecuencia, *TPDV* vuelve a ser un buen candidato.

En conclusión, tenemos dos candidatos:

- *TPDV*
- *Venta*

Consideremos algunas de las consecuencias de estas posibilidades respecto a los patrones Alta Cohesión y Bajo Acoplamiento de GRASP. Si escogemos la *Venta* para crear el *Pago*, el trabajo (o las responsabilidades) de *TPDV* será más ligero, pues dará origen a una definición más simple de *TPDV*. Además no necesita saber que existe una instancia *Pago*, porque podemos registrarla indirectamente mediante la *Venta* y obtendremos así un menor acoplamiento con *TPDV*. Ello, a su vez, nos permite construir el diagrama de colaboración de la figura 19.10.

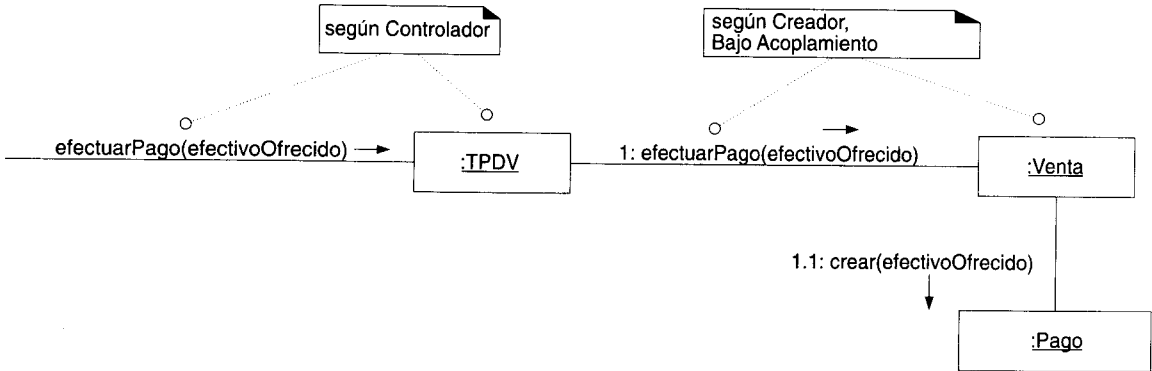


Figura 19.10 Diagrama de colaboración TPDV-efectuarPago.

Este diagrama de colaboración cumple con las poscondiciones del contrato: se creó el *Pago* asociado a *Venta* y se estableció su *montoOfrecido*.

19.7.3 Registro de la venta

Una vez formulados, los requerimientos estipulan que la venta debería ser colocada en un registro histórico o bitácora. Como de costumbre, el patrón Experto tendría que ser el primero en considerarse, salvo que se trate de un problema de controlador o de creación (pero no es así); además habría que expresar la responsabilidad:

¿Quién asume la responsabilidad de conocer todas las ventas registradas y de llevar a cabo el registro?

Es lógico que una *Tienda* conozca todas las ventas registradas, pues están estrechamente relacionadas con sus finanzas. Entre otras opciones mencionamos la inclusión de los conceptos clásicos de la contabilidad, como un *LibroMayordeVentas*. Es conveniente usar un objeto *LibroMayordeVentas* conforme vaya creciendo el diseño y la *Tienda* vaya perdiendo cohesión (figura 19.11).

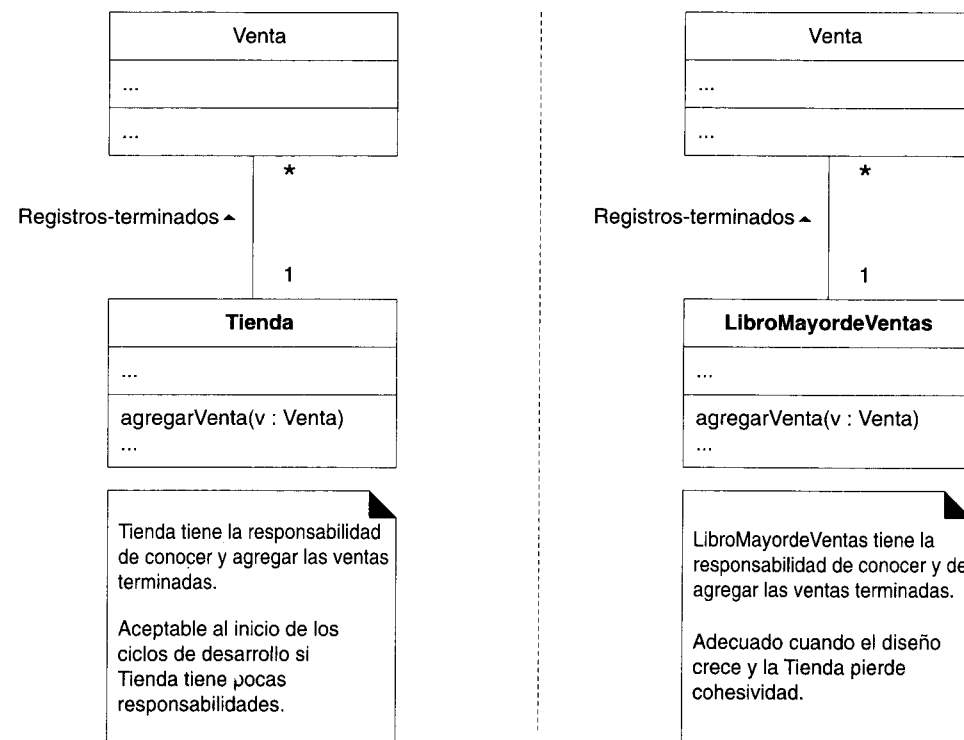


Figura 19.11 ¿Quién asumirá la responsabilidad de conocer las ventas terminadas?

Nótese asimismo que las poscondiciones del contrato indican relacionar la *Venta* con la *Tienda*. Éste es un ejemplo donde las poscondiciones tal vez no sean lo que en ver-

dad deseamos conseguir con el diseño. Acaso no hayamos pensado antes en un *LibroMayordeVentas*, pero ahora que lo hicimos hemos decidido utilizarlo en vez de una *Tienda*. De ser así, en teoría incorporaríamos también *LibroMayordeVentas* al modelo conceptual. Cabe esperar este tipo de descubrimiento y cambio durante la fase de diseño. Por fortuna, el desarrollo iterativo ofrece un ciclo de vida para el cambio continuo.

En este caso, no abandonaremos el plan original de usar la *Tienda*.

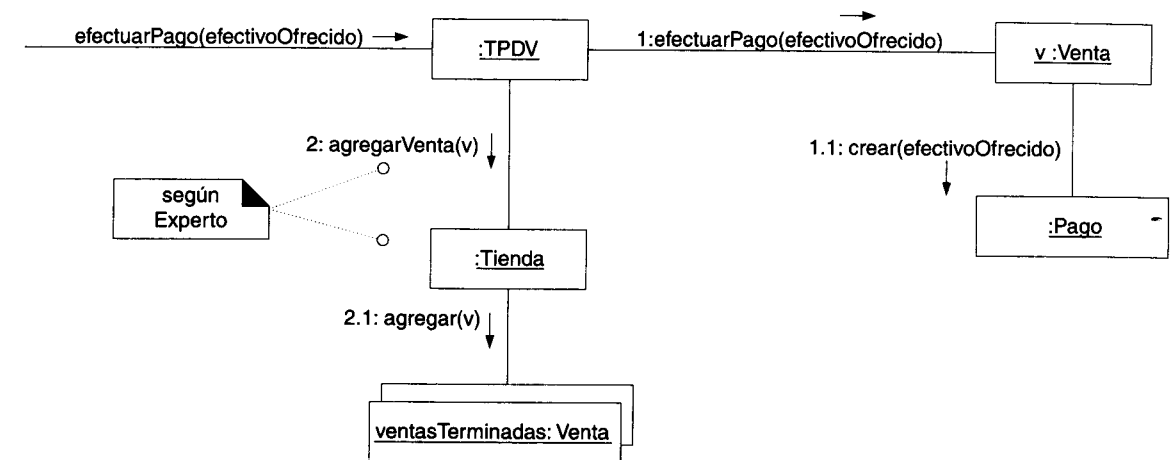


Figura 19.12 Registro de una venta terminada.

19.7.4 Cálculo del saldo

La sección dedicada a las responsabilidades estipula que el saldo ha sido calculado; entonces se imprimirá en un recibo y se mostrará en un monitor.

Debido al principio de Separación Modelo-Vista, no debe preocuparnos cómo se presentará visualmente el saldo, pero sí es necesario asegurarnos de que se conozca. Adviértase que ninguna clase lo conoce en este momento; por ello, mediante un diagrama de colaboración hay que construir un diseño de las interacciones de los objetos que cumpla con este requerimiento.

Como es habitual, el patrón Experto será el primero en considerarse, salvo que se trate de un problema de controlador o de creación (pero no es así), y habrá que formular la responsabilidad en los siguientes términos:

¿Quién asume la responsabilidad de conocer el saldo?

Para calcular el saldo necesitamos el total de la venta y el pago en efectivo ofrecido. Por tanto, *Venta* y *Pago* son Expertos parciales en la solución de este problema.

Si el *Pago* es el principal responsable de conocer el saldo, necesitará ser visible a la *Venta* para pedirle su total. Como por ahora no la conoce, con este método aumentará el acoplamiento global del diseño: no soportará el patrón Bajo Acoplamiento.

En cambio, si la *Venta* es el principal responsable de conocer el saldo, necesitará ser visible al *Pago* para pedirle su efectivo ofrecido. Dado que la *Venta* ya es visible al *Pago* —como su creador—, con este enfoque no se aumenta el acoplamiento global; de ahí que sea un diseño preferible.

En consecuencia, el diagrama de colaboración de la figura 19.13 ofrece una solución para conocer el saldo.

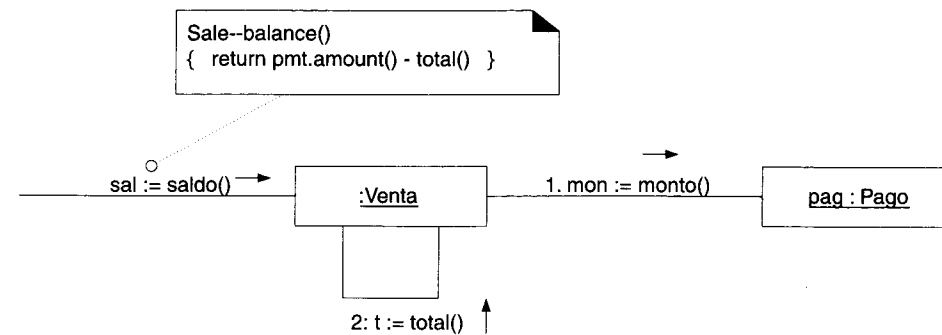


Figura 19.13 El diagrama de colaboración venta-saldo.

19.8 El diagrama de colaboración: Iniciar

19.8.1 ¿Cuándo crear el diagrama de colaboración Iniciar?

La mayoría de los sistemas, si no es que todos, tienen un caso de uso *Iniciar* y alguna operación inicial relacionada con el comienzo de la aplicación. Aunque es la primera en ser ejecutada, se ha considerado la conveniencia de posponer la preparación del respectivo diagrama de colaboración hasta después de considerar el resto de las operaciones del sistema. Con ello se garantiza el descubrimiento de información importante referente a las actividades de inicialización que se requieren para brindar soporte a los diagramas ulteriores de interacción.

Prepare al final el diagrama de colaboración *Iniciar*.

19.8.2 Cómo se inician las aplicaciones

La operación *Iniciar* representa en forma abstracta la fase inicializadora de la ejecución cuando se comienza una aplicación. Si queremos saber cómo diseñar un diagrama de colaboración para esa operación, conviene conocer los contextos donde puede ocurrir la inicialización. Depende del lenguaje de programación y del sistema operativo la manera en que comienza una aplicación y en que se inicializa.

En todos los casos, una expresión de diseño común consiste en crear al final un **objeto inicial del dominio**: el primero del dominio del problema que generamos. En su método de inicialización (un constructor en Java, por ejemplo), es entonces el encargado de producir el resto de los objetos del dominio del problema y de inicializarlos.

El lugar donde se crea el objeto inicial del dominio dependerá del lenguaje de programación orientado a objetos y también del sistema operativo. Así, en un applet de Java su inicio puede hacer que los atributos (variables de instancia) sean instanciados. Uno de ellos puede ser el objeto inicial del dominio; por ejemplo una *Tienda* que crea otros objetos como una *TPDV*.

```

public class TPDVApplet extends Applet
{
    public void init()
    {
        tpdv = tienda.obtenerTPDV();
    }

    // Tienda es el objeto inicial del dominio.
    // El constructor de Tienda crea
    // otros objetos del dominio
    private Tienda tienda = new Tienda();

    private TPDV tpdv;
    private Venta venta;
}

```

19.8.3 Interpretación de la operación del sistema Iniciar

La exposición anterior sobre las formas en que comienza una aplicación muestra que la operación del sistema *Iniciar* es una abstracción independiente del lenguaje. En la fase de diseño se observa variación en el sitio donde se produce el objeto inicial y en el hecho de que asuma o no el control del proceso. El objeto inicial del dominio no suele hacerlo si hay una interfaz gráfica para el usuario, pero sí en caso contrario.

El diagrama de colaboración de la operación *Iniciar* describe lo que sucede cuando se crea el objeto inicial del dominio del problema y, opcionalmente, lo que sucede si asume el control. No incluye ninguna actividad anterior ni subsecuente en la capa de objetos destinada al interfaz gráfico para el usuario, si es que existe.

Por tanto, la operación *Iniciar* puede interpretarse así:

1. En un diagrama de colaboración, envíe un mensaje *crear()* para producir el objeto inicial del dominio.
2. (opcional) Si el objeto inicial va a asumir el control del proceso, en un segundo diagrama de colaboración envíele un mensaje *ejecutar* (u otro equivalente).

19.8.4 Aplicación de la operación Iniciar al punto de venta

La operación sistémica *Iniciar* ocurre cuando un supervisor enciende el sistema de punto de venta y el software se ejecuta. Supongamos que contamos con una interfaz gráfica para el usuario y que un objeto de la capa de presentación (por ejemplo, una instancia applet de Java) se encargará de producir el objeto inicial del dominio del problema.¹ Supongamos asimismo que el objeto inicial *no* asume el control del proceso; éste permanece en el applet tras haber sido creado el objeto inicial del dominio. Así pues, el diagrama de colaboración de la operación *Iniciar* puede reinterpretarse exclusivamente como un mensaje *crear()* que se envía para producir el objeto inicial.

19.8.5 Elección del objeto inicial del dominio

¿Cuál debería ser la clase del objeto inicial del dominio?

Escoja como objeto inicial del dominio:

■ Una clase que represente todo el sistema de información lógico.

■ Una clase que represente íntegramente el negocio u organización.

En la selección de estas opciones pueden influir consideraciones referentes a los patrones Alta Cohesión y Bajo Acoplamiento. Por tanto, en la aplicación del punto de venta, entre las decisiones razonables acerca del objeto inicial se encuentran:

Todo el Sistema de información lógico	TPDV, SistemaInformativoMenudeo
Negocio u organización global	Tienda

En esta aplicación se escogió la *Tienda* como objeto inicial.

19.8.6 Objetos persistentes: EspecificacioneProducto

En la aplicación realista, las instancias *EspecificacioneProducto* residirán en un medio de almacenamiento persistente: una base de datos relacional o de objetos. Durante la operación *Iniciar*, si no hay más que unos cuantos de esos objetos, podemos cargarlos todos en la memoria directa de la computadora. Pero si hay muchos, cargarlos todos

¹ En un caso real, probablemente no se trate de un applet de Java, pero cuando se sirve de ejemplo ofrece un estándar simple y muy conocido.

requeriría demasiada memoria o tiempo. Una alternativa —quizá la más viable— consiste en cargar las instancias individuales en la memoria conforme vaya necesitándose. El diseño de cómo, previa solicitud, cargar dinámicamente los objetos y almacenarlos en la memoria a partir de una base de datos es simple si se utiliza una base de datos de objetos, pero difícil si se utiliza una base de datos relacional. Por ahora vamos a posponer la solución de este problema y haremos la suposición simplificadora de que todas las instancias *EspecificacioneProducto* pueden crearse “mágicamente” en la memoria por medio del objeto *CatalogodeProductos*.

En el capítulo 38 examinaremos el tema de los objetos persistentes y la manera de cargarlos en la memoria.

19.8.7 El diagrama de colaboración guardar--crear() (store-create)

He aquí el contrato *Iniciar*:

Contrato	
Nombre:	Iniciar().
Responsabilidades:	Inicializar el sistema.
Tipo:	Sistema.
Referencias cruzadas:	
Notas:	
Excepciones:	
Salida:	
Precondiciones:	
Post-condiciones:	
■ Se crearon <i>Tienda</i> , <i>TPDV</i> , <i>CatalogodeProductos</i> y <i>EspecificacionesdeProducto</i> (creación de instancia).	
■ Se asoció <i>CatalogodeProductos</i> a <i>EspecificacionesdeProducto</i> (asociación formada).	
■ Se asoció <i>Tienda</i> a <i>CatalogodeProductos</i> (asociación formada).	
■ Se asoció <i>Tienda</i> a <i>TPDV</i> (asociación formada).	
■ Se asoció <i>TPDV</i> a <i>Catálogo de productos</i> (asociación formada).	

Como hemos señalado, lo anterior se interpretará como lo que sucede a raíz de la creación del objeto inicial —la *Tienda*— cuando se envía un mensaje *crear*.

En la figura 19.14 se incluye un diagrama de colaboración que cumple con estas poscondiciones. La *Tienda* fue elegida para crear *CatalogodeProductos* y *TPDV* mediante el patrón Creador. También se eligió *CatalogodeProductos* para producir *EspecificacionesdeProducto*.

Observe que la creación de todas las instancias *EspecificaciondeProducto* y su incorporación a un contenedor se realizan en una sección de repetición, indicada por un asterisco (*) colocado después de los números de secuencia.

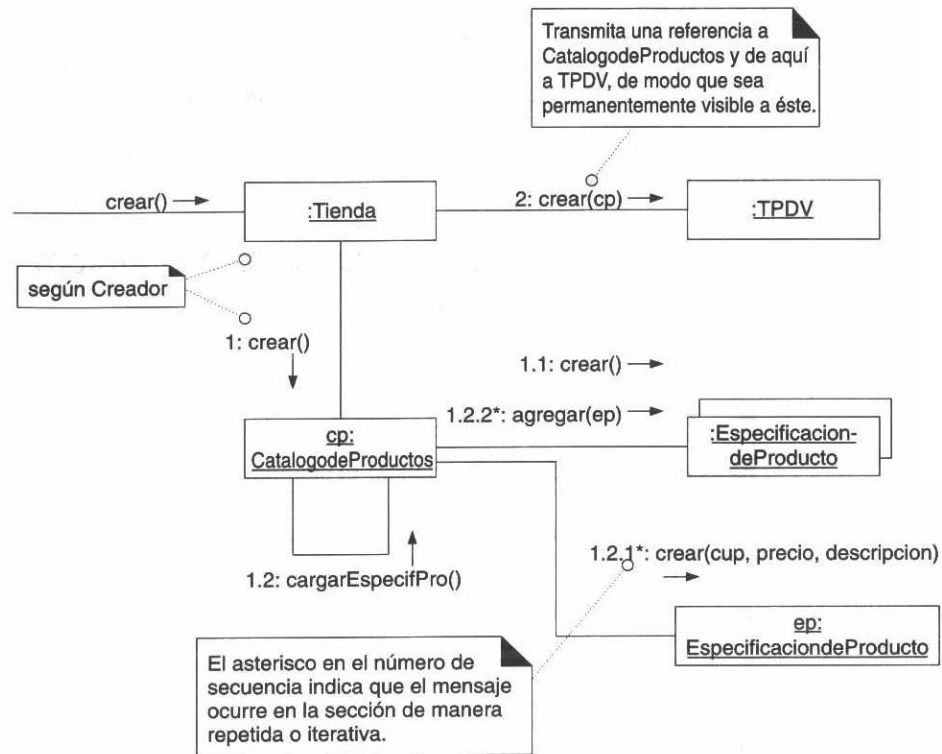


Figura 19.14 Creación del objeto inicial del dominio y de objetos posteriores.

Una discrepancia interesante entre el análisis y el diseño se ejemplifica en el hecho de que la *Tienda* sólo crea *un* objeto *TPDV*. Desde el punto de vista analítico, una tienda real puede albergar *muchas terminales* en el punto de venta. Pero ahora estamos considerando un diseño de software, no lo que ocurre en el mundo real. La *Tienda* representada en un diagrama de colaboración no es una tienda real; es un objeto de software. El objeto *TPDV* de un diagrama tampoco es una terminal real; es un objeto de software. En nuestros requerimientos actuales, el software *Tienda* no necesita crear más que una sola instancia del software *TPDV*.

En la fase de análisis y de diseño, la multiplicidad entre las clases de objetos tal vez no sea la misma.

19.9

Cómo conectar la capa de presentación y la de dominio

Como hemos visto, si interviene una interfaz gráfica para el usuario —digamos un applet de Java que forma parte del coordinador de aplicaciones y de la capa de presentación—, el applet se encargará de comenzar la producción del objeto inicial del dominio, como se aprecia en la figura 19.15. Primero, el applet crea el objeto inicial del dominio (una instancia *Tienda*), que a su vez produce una instancia *TPDV*. Luego solicita a la *Tienda* una referencia a la instancia *TPDV* y la guarda en un atributo, para que el applet pueda enviarle directamente un mensaje.

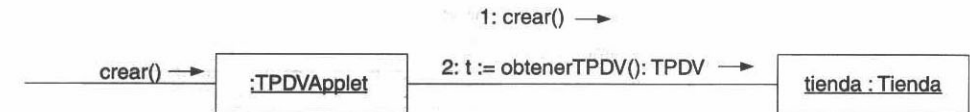


Figura 19.15 Conexión de las capas de presentación y de dominio.

Una vez que el applet tiene una conexión con la instancia *TPDV*, puede enviarle mensajes de eventos del sistema, entre ellos *introducirProducto* y *terminarVenta* (figura 19.16).

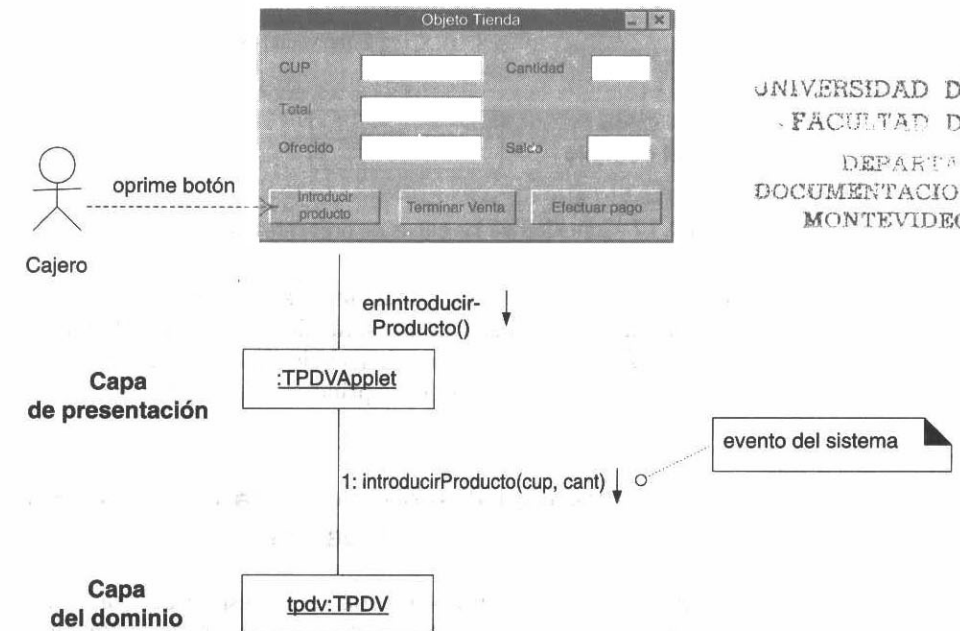


Figura 19.16 Conexión de las capas de presentación y dominio.

En el caso del mensaje *introducirProducto*, la ventana habrá de exhibir el total actual después de cada captura. Como se ve en la figura 19.17, después que *TPDVApplet* envía el mensaje *introducirProducto* al objeto *TPDV*:

1. Obtiene una referencia a la *Venta* (si es que todavía no tiene una).
2. Almacena la referencia *Venta* en un atributo.
3. Envía un mensaje de *total* a la *Venta* con el fin de conseguir la información necesaria para presentar en la ventana el total actual.

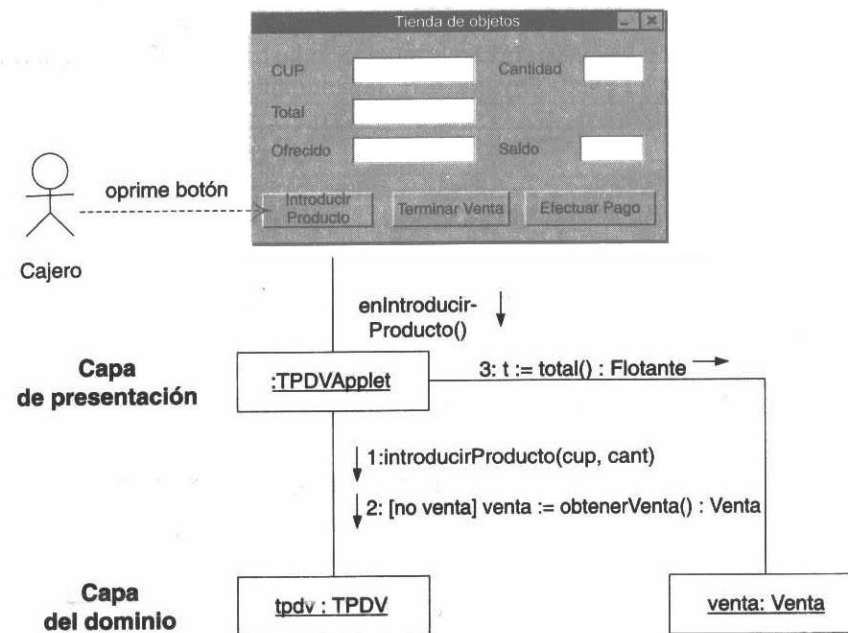


Figura 19.17 Conexión de las capas de presentación y dominio.

Adviértase que, en estos diagramas de colaboración, el applet de Java (*TPDVApplet*), que forma parte de la capa de presentación, no maneja la lógica de la aplicación. A través de *TPDV* envía peticiones de trabajo (las operaciones del sistema) a la capa del dominio. Esto nos lleva al siguiente principio de diseño:

Responsabilidades de las capas de presentación y de dominio

La capa de presentación no debería tener responsabilidades lógicas de dominio. Tan sólo debería encargarse de las tareas de presentación (interfaz), entre ellas actualizar los elementos contenidos en la ventana.

La capa de presentación debería enviarle a la capa del dominio las peticiones de las tareas orientadas al dominio, que es la que se ocupa de ellas.

19.10 Resumen

El diseño de las interacciones de los mensajes y la asignación de responsabilidades constituyen la esencia de un diseño orientado a objetos. Las decisiones que se tomen en él ejercerán un impacto muy profundo sobre la extensibilidad, la claridad y la capacidad de mantenimiento de un sistema de software orientado a objetos, así como en el grado y la calidad de los componentes reutilizables. Contamos con algunos principios que nos permiten tomar decisiones respecto a la asignación de responsabilidades. Los patrones GRASP resumen algunos de los más generales y comunes de los cuales se sirven los diseñadores orientados a objetos.

En este capítulo hemos examinado la aplicación de dichos patrones para crear clases bien diseñadas y un sistema de objetos que interactúan.

DETERMINACIÓN DE LA VISIBILIDAD

Objetivos

- Identificar cuatro tipos de visibilidad.
- Diseñar para lograr la visibilidad.
- Dar ejemplos de los tipos de visibilidad en la notación de UML.

20.1 Introducción

La visibilidad es la capacidad de un objeto para ver otro o hacer referencia a él. En el presente capítulo estudiaremos los problemas de diseño relacionados con la visibilidad.

20.2 Visibilidad entre objetos

Los diagramas de colaboración creados para los eventos del sistema (*introducirProducto*, por ejemplo) describen gráficamente los mensajes entre objetos. Para que un objeto emisor envíe un mensaje a un objeto receptor, el emisor tiene que ser visible a éste: debe tener alguna clase de referencia o apuntador a él.

Por ejemplo, el mensaje *especificacion* enviado de una instancia *TPDV* a una instancia *CatalogodeProductos* significa que ésta es visible a aquélla, como se muestra en la figura 20.1.

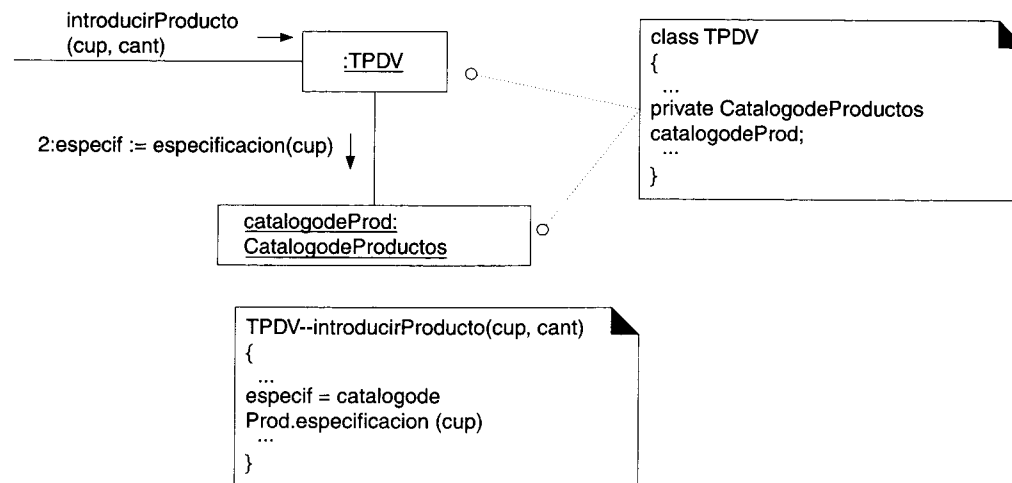


Figura 20.1 Se requiere visibilidad de la instancia TPDV a CatalogodeProductos.¹

Cuando se construye un diseño de objetos que interactúan, es preciso asegurarse de que exista la visibilidad necesaria pues de otro modo no podrá darse soporte a la interacción de los mensajes.

El UML cuenta con una notación especial para mostrar la visibilidad; en este capítulo explicaremos varios tipos de visibilidad y su representación gráfica.

20.3 Visibilidad

En el lenguaje cotidiano, la **visibilidad** es la capacidad de un objeto para “ver” o hacer referencia a otro. En un sentido más general, se relaciona con la cuestión del alcance o ámbito: ¿es un recurso (digamos una instancia) dentro del ámbito de otro? Hay cuatro formas comunes en que podemos conseguir la visibilidad del objeto *A* al objeto *B*:

1. **Visibilidad de atributos:** B es un atributo de A.
2. **Visibilidad de parámetros:** B es un parámetro de un método de A.
3. **Visibilidad declarada localmente:** se declara que B es un objeto local en un método de A.
4. **Visibilidad global:** en alguna forma B es visible globalmente.

¹ En este ejemplo de código y en otros subsecuentes, puede simplificarse el lenguaje por razones de brevedad y de claridad.

Consideramos la visibilidad por lo siguiente:

Para que un objeto A envíe un mensaje a un objeto B, debe ser visible a aquél.

Por ejemplo, para preparar un diagrama de colaboración donde se envíe un mensaje de la instancia *TPDV* a la instancia *CatalogodeProductos*, la primera deberá tener visibilidad de la segunda. Una solución usual consiste en que una referencia a la instancia *CatalogodeProductos* sea conservada como un atributo de *TPDV*.

20.3.1 Visibilidad de atributos

Existe **visibilidad de atributos** de A a B cuando B es un atributo de A. Se trata de una visibilidad relativamente permanente porque persiste mientras existan A y B. Es una forma muy común de visibilidad en los sistemas orientados a objetos.

Por ejemplo, en una definición de *TPDV* en una clase de Java, esta instancia puede tener visibilidad de atributo para un *CatalogodeProductos* por ser un atributo (variable de instancia en Java) de *TPDV*.

```

public class POST
{
    ...
    private ProductCatalog prodCatalog;
    ...
}
  
```

Se requiere esta visibilidad porque, en el diagrama de colaboración *introducirProducto* de la figura 20.2, se necesita una instancia *TPDV* que envíe el mensaje *especificacion* a una instancia *CatalogodeProductos*.

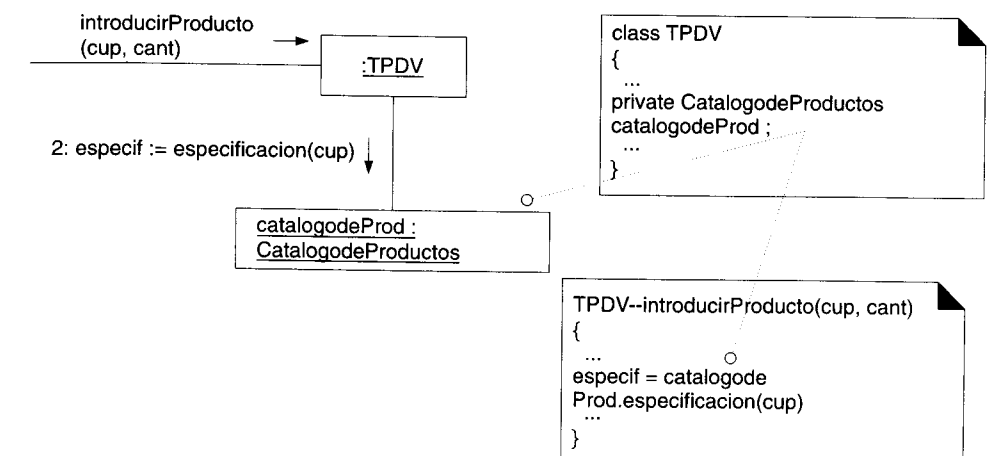


Figura 20.2 Visibilidad de atributos.

20.3.2 Visibilidad de los parámetros

Existe **visibilidad de parámetros** de A a B cuando B se transmite como un parámetro a un método de A. Se trata de una visibilidad relativamente temporal porque persiste sólo dentro del ámbito del método. Después de la visibilidad de atributos, es la segunda forma más común de visibilidad en los sistemas orientados a objetos.

He aquí un ejemplo: cuando el mensaje *hacerLineadeProducto* se envía a una instancia *Venta*, una instancia *EspecificaciondeProducto* es transferida como parámetro. Dentro del ámbito del método *hacerLineadeProducto*, la *Venta* tiene visibilidad de parámetro para *EspecificaciondeProducto* (figura 20.3).

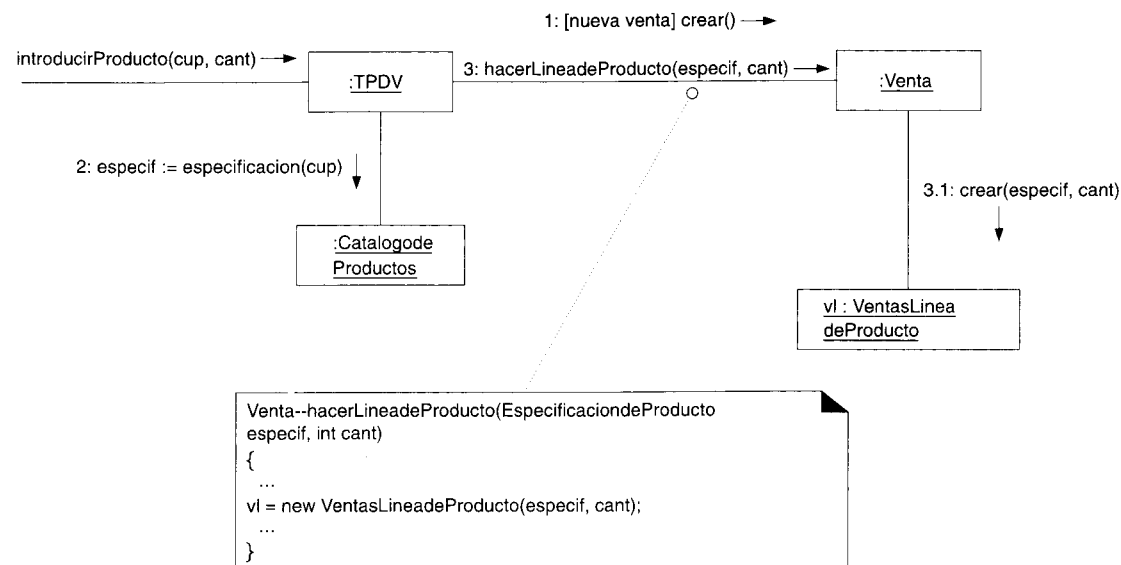


Figura 20.3 Visibilidad de parámetros.

Con frecuencia la visibilidad de parámetros se transforma en visibilidad de atributos. Así, cuando la *Venta* crea una nueva instancia *VentasLineadeProducto*, transmite una *EspecificaciondeProducto* a su método de inicialización (en C++ o en Java sería su **constructor**). Dentro del método de inicialización, se asigna el parámetro a un atributo y con ello se logra la visibilidad de atributos (figura 20.4).

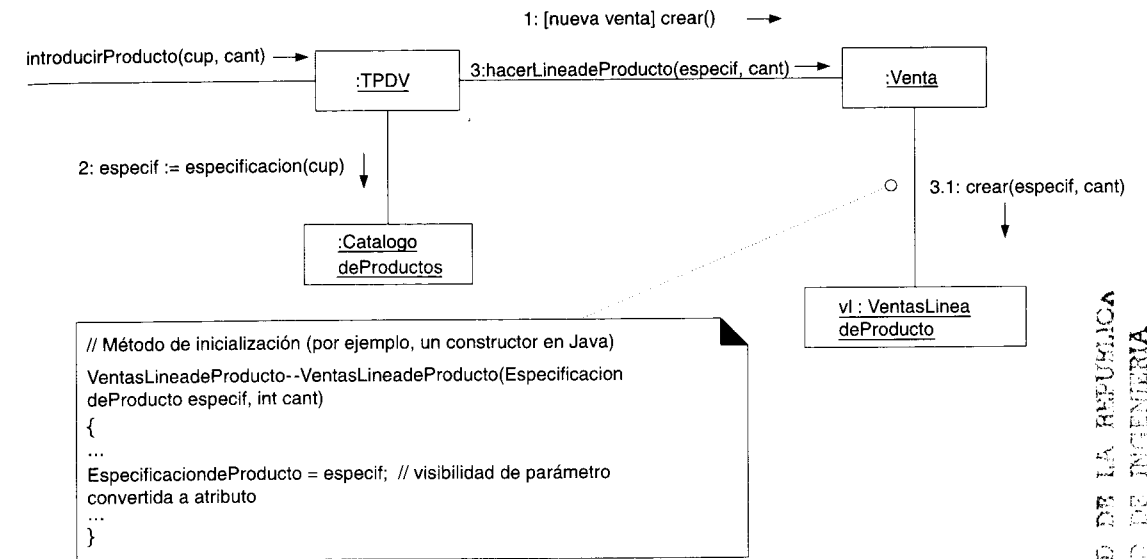


Figura 20.4 Transformación de visibilidad de parámetros a visibilidad de atributos.

20.3.3 Visibilidad declarada localmente

Existe una **visibilidad declarada localmente** de A a B, cuando se declara que B es un objeto local dentro de un método de A. Se trata de una visibilidad relativamente temporal porque persiste sólo dentro del ámbito del método. Después de la visibilidad de parámetros, es la tercera forma más común de visibilidad en los sistemas orientados a objetos. A continuación se mencionan dos medios habituales con que se logra esta clase de visibilidad:

1. Crear una nueva instancia local y asignarla a una variable local.
2. Asignar a una variable local el objeto devuelto proveniente de la llamada a un método.

Se produce una variación de (2) cuando el método no declara de manera explícita una variable, pero existe implícitamente una como resultado de un objeto devuelto procedente de la invocación de un método.

Igual que en la visibilidad de parámetros, a menudo la visibilidad declarada localmente es transformada en visibilidad de atributo.

Un ejemplo de la segunda variación se encuentra en el método *introducirProducto* de la clase *TPDV* (figura 20.5).

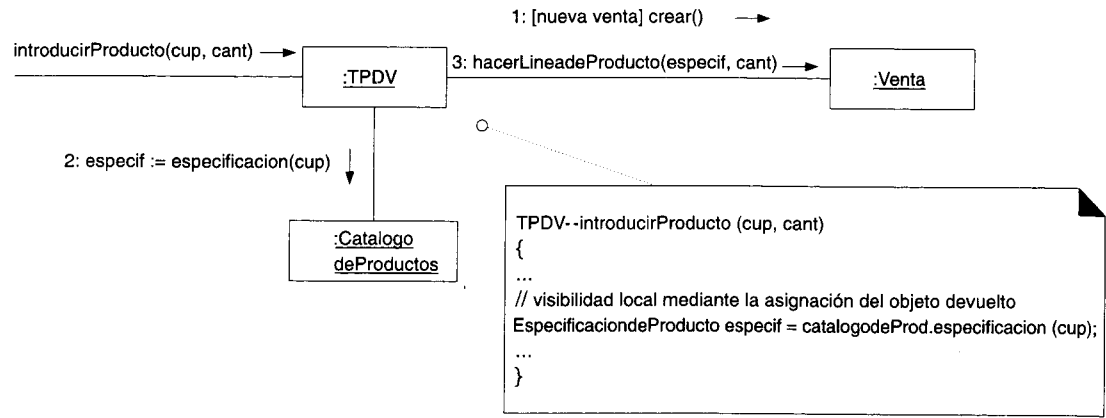


Figura 20.5 Visibilidad local.

20.3.4 Visibilidad global

Existe **visibilidad global** de A a B cuando B es global para A. Se trata de una visibilidad relativamente permanente, porque persiste mientras existan A y B. Es el tipo de visibilidad menos frecuente en los sistemas orientados a objetos.

La forma más obvia —aunque menos conveniente— de alcanzar la visibilidad global consiste en asignar una instancia a una variable global.

El método usual de conseguir la visibilidad global es utilizar el patrón **Singleton** [GHJV95], del cual nos ocuparemos en el capítulo 35.

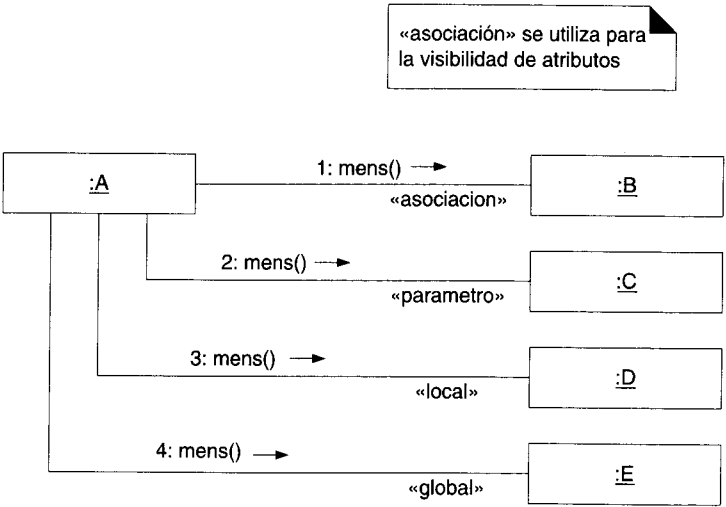


Figura 20.6 Estereotipos de implementación para la visibilidad.

20.4 Presentación de la visibilidad en el UML

El lenguaje UML cuenta con una notación que describe la implementación de varios tipos de visibilidad en un diagrama de colaboración (figura 20.6). Estos elementos decorativos son opcionales y normalmente no se requieren; son de gran utilidad cuando hace falta la clarificación.

DIAGRAMAS DE CLASES DEL DISEÑO

Objetivos

- Crear diagramas de clases del diseño.
- Identificar las clases, los métodos y las asociaciones que deben incluirse en un diagrama de clases de este tipo.

21.1 Introducción

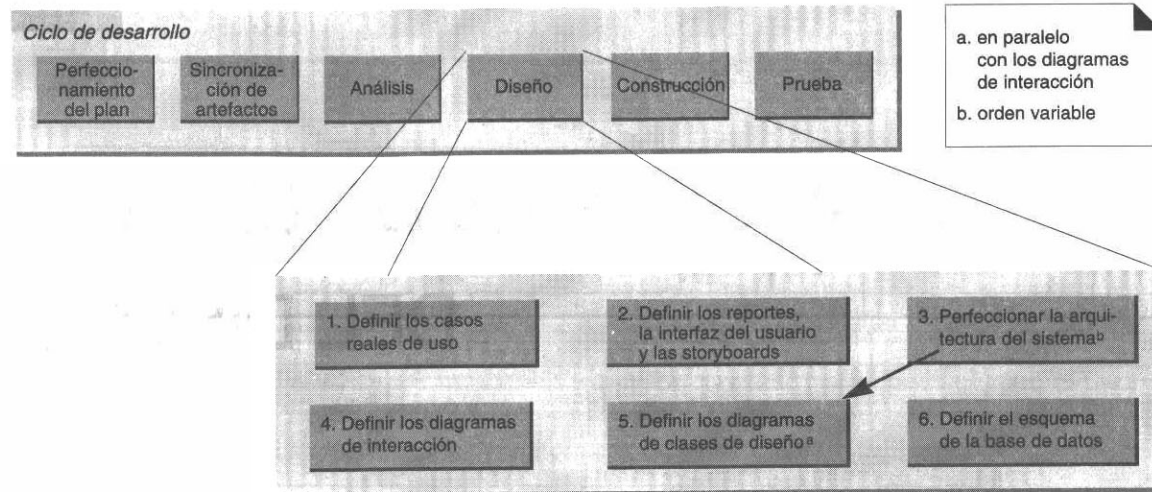
Una vez terminados los diagramas de interacción para el ciclo actual de desarrollo de la aplicación de punto de venta, podemos identificar la especificación de las clases de software (y las interfaces) que participan en la solución de software y complementarlas con detalles de diseño, por ejemplo los métodos.

El lenguaje UML ofrece una notación que muestra los detalles de diseño en los diagramas de estructura —o clase— estática; en este capítulo vamos a estudiarla y a elaborar los diagramas de clases del diseño.

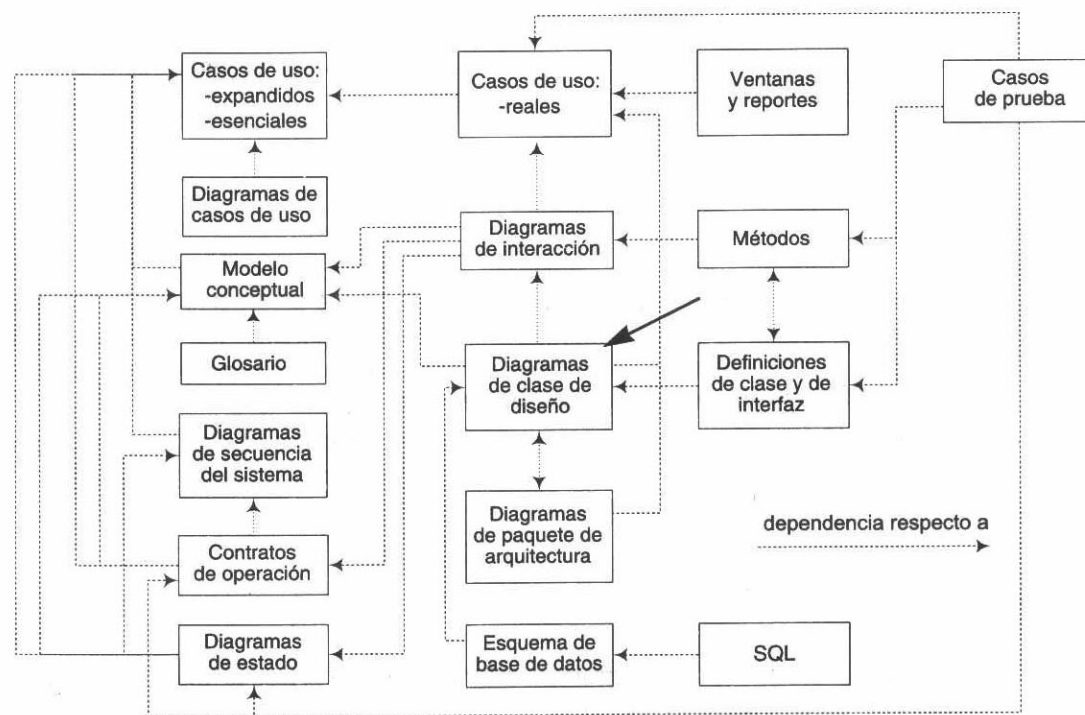
21.2 Actividades y dependencias

La definición de este tipo de diagramas se lleva a cabo en la fase de diseño del ciclo de desarrollo. Su preparación exige crear antes:

- Diagramas de interacción: a partir de ellos el diseñador identifica las clases de software que intervienen en la solución, así como los métodos de las clases.
- Modelo conceptual: a partir de éste el diseñador agrega detalles a la definición de las clases.



Actividades de la fase de diseño dentro de un ciclo de desarrollo.



Dependencias de los artefactos durante la fase de construcción.

21.3 Cuándo crear diagramas de clases del diseño

Aunque nuestra exposición de los diagramas de clases del diseño *viene después* de su elaboración, en la práctica suelen prepararse al mismo tiempo que los diagramas de interacción. Podemos bosquejar muchas clases, nombres de métodos y relaciones al inicio de la fase de diseño, aplicando los patrones de asignación de responsabilidades antes de dibujar los diagramas de interacción. Frente a las tarjetas CRC son una notación alterna de carácter más gráfico, que sirven para registrar las responsabilidades y los colaboradores.

21.4 Ejemplo de un diagrama de clases del diseño

El diagrama de la figura 21.1 contiene una definición parcial de software de las clases *TPDV* y *Venta*.

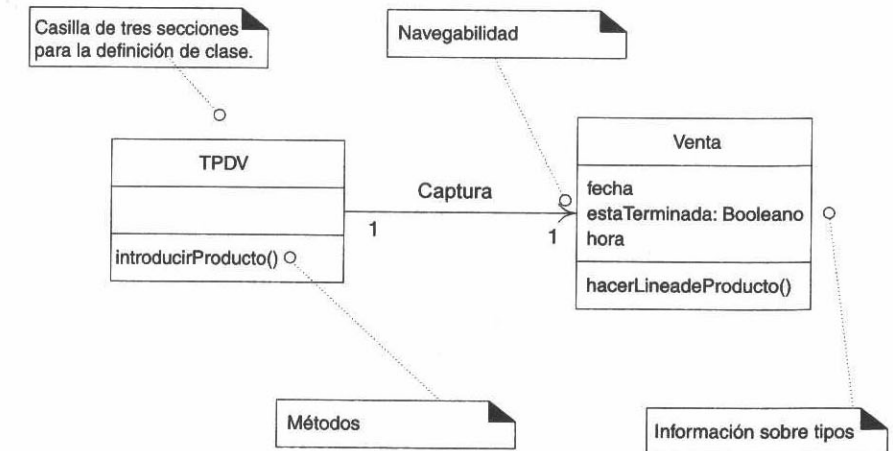


Figura 21.1 Ejemplo de diagrama de clases del diseño.

Además de las asociaciones y de los atributos básicos, se ha ampliado el diagrama para incluir —por ejemplo— los métodos de cada clase, la información sobre el tipo de los atributos, su visibilidad y la navegación entre objetos.

21.5 Diagramas de clases del diseño

El **diagrama de clases del diseño** describe gráficamente las especificaciones de las clases de software y de las interfaces (las de Java, por ejemplo) en una aplicación. Normalmente contiene la siguiente información:

- clases, asociaciones y atributos
- interfaces, con sus operaciones y constantes
- métodos
- información sobre los tipos de los atributos
- navegabilidad
- dependencias

A diferencia del modelo conceptual, un diagrama de este tipo contiene las definiciones de las entidades del software en vez de conceptos del mundo real. El UML no define concretamente un elemento denominado “diagrama clases del diseño”, sino que se sirve de un término más genérico: “diagrama de clases”. Yo opté por incluir el término “diseño” en “diagrama de clases” para recalcar que se trata de una perspectiva desde el punto de vista del diseño de las entidades de software y no de una concepción analítica sobre los conceptos del dominio.

21.6 Cómo elaborar un diagrama de clases del diseño

Aplique la siguiente estrategia para construir un diagrama de clases del diseño:

Para preparar un diagrama de clases orientado al diseño:

1. Identifique todas las clases que participan en la solución del software. Para ello analice los diagramas de interacción.
2. Dibújelas en un diagrama de clases.
3. Duplique los atributos provenientes de los conceptos asociados del modelo conceptual.
4. Agregue los nombres de los métodos analizando los diagramas de interacción.
5. Incorpore la información sobre los tipos a los atributos y a los métodos.
6. Agregue las asociaciones necesarias para dar soporte a la visibilidad requerida de los atributos.
7. Agregue flechas de navegabilidad a las asociaciones para indicar la dirección de la visibilidad de los atributos.
8. Agregue las líneas de relaciones de dependencia para indicar la visibilidad no relacionada con los atributos.

21.7 Comparación entre el modelo conceptual y los diagramas de clases del diseño

En el modelo conceptual una *Venta* no representa una definición de software; más bien es una abstracción de un concepto del mundo real acerca del cual queremos afirmar algo. En cambio, los diagramas de clases del diseño expresan —para el sistema computarizado— la definición de clases como componentes del software. En estos diagramas una *Venta* representa una clase de software (figura 21.2).

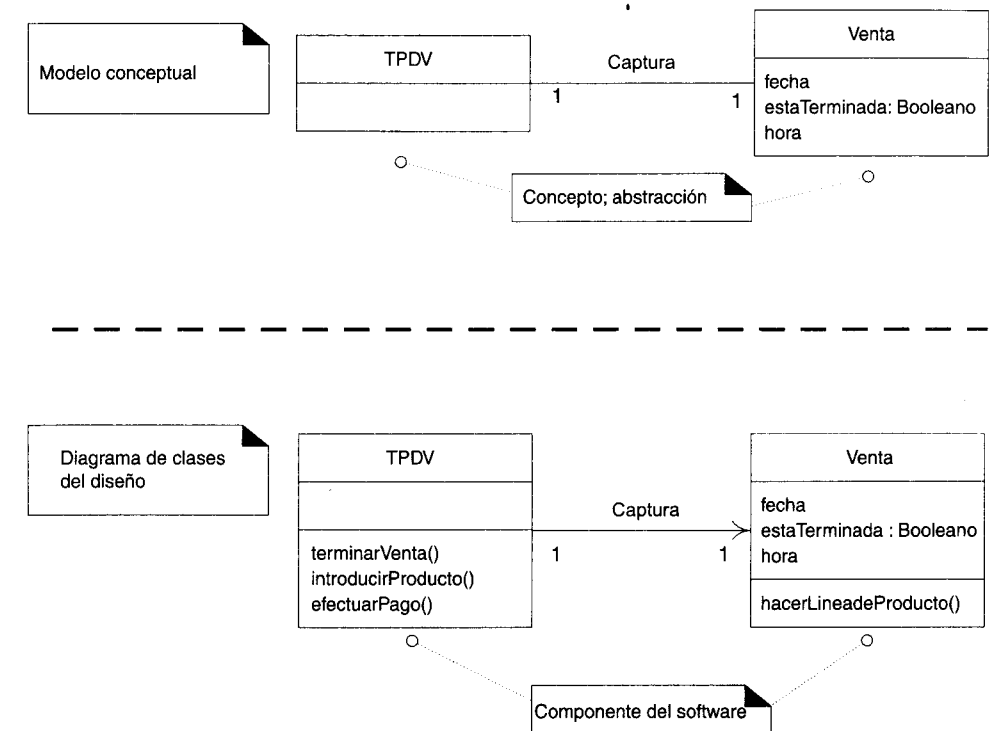


Figura 21.2 Comparación entre el modelo conceptual y el diagrama de clases del diseño.

21.8 Creación de diagramas de clases del diseño para el punto de venta

21.8.1 Identificación de las clases del software y su ilustración

El primer paso en la preparación de estos diagramas como parte del modelo de la solución consiste en identificar las clases que intervienen en la solución del software.

Podemos encontrarlas con sólo examinar todos los diagramas de interacción y listar las clases mencionadas. En las aplicaciones de TPDV ellas son:

TPDV	Venta
CatalogodeProductos	EspecificaciondeProductos
Tienda	VentasLineadeProducto
Pago	

El siguiente paso consiste en dibujar un diagrama de clases para estas clases e incluir en el modelo conceptual los atributos ya identificados (figura 21.3).

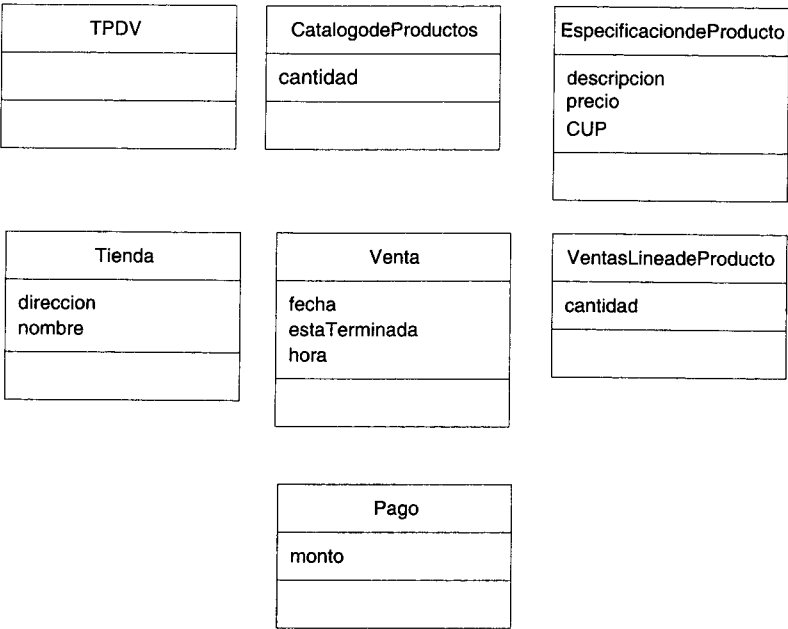


Figura 21.3 Clases de software en la aplicación.

Nótese que muchos de los conceptos del modelo conceptual, entre ellos *Cajero*, *Gerente* y *Producto*, no aparecen en el diseño. No es necesario —en el actual ciclo de desarrollo— representarlos en el software. Pero en ciclos ulteriores podemos incorporarlos al diseño, conforme vayamos descubriendo nuevos requerimientos y casos de uso.

1.8.2 Agregar los nombres de los métodos

Para identificar los métodos de las clases se analizan los diagramas de colaboración. Por ejemplo, si el mensaje *hacerLineadeProducto* se envía a una instancia de la clase *Venta*, entonces ésta deberá definir el método *hacerLineadeProducto* (figura 21.4).

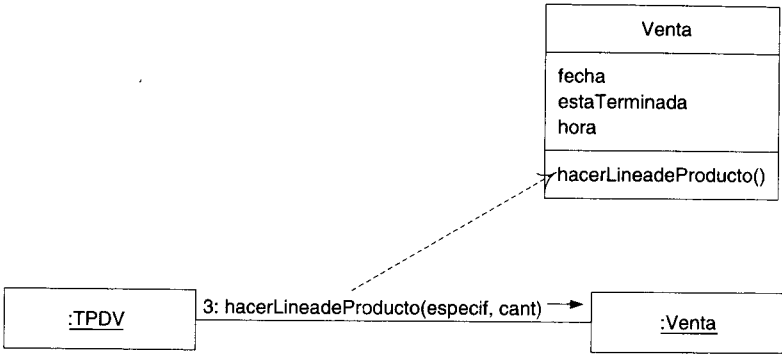


Figura 21.4 Nombres de los métodos tomados de los diagramas de colaboración.

En términos generales, el conjunto de los mensajes enviados a la clase X a través de los diagramas de colaboración indica la mayoría de los métodos que ha de definir la clase X.

Al observar detenidamente todos los diagramas de colaboración referentes a la aplicación del punto de venta, obtenemos la asignación de los métodos que aparecen en la figura 21.5.

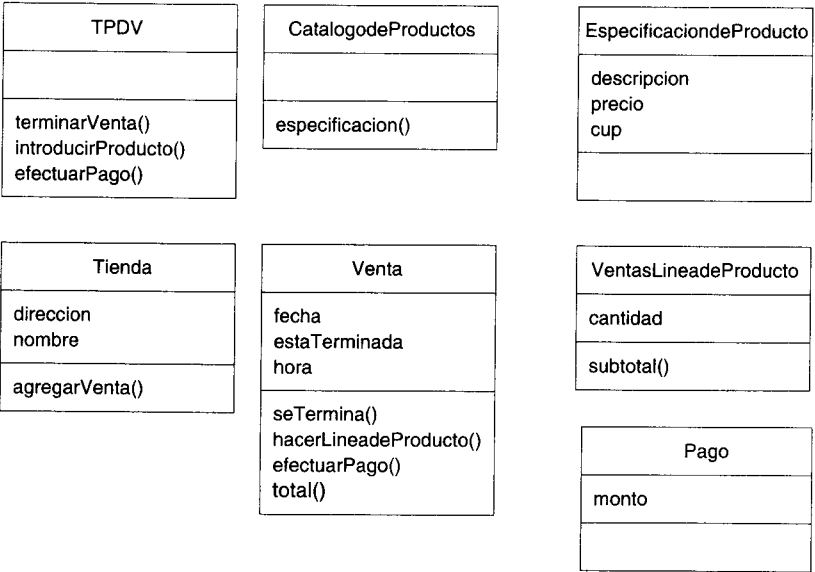


Figura 21.5 Métodos en la aplicación.

21.8.3 Nombres de los métodos: aspectos

Los siguientes aspectos especiales deben tenerse en cuenta respecto a los nombres de los métodos:

- Interpretación del mensaje *crear()*
- Descripción de los métodos de acceso
- Interpretación de los mensajes dirigidos a multiobjetos
- Sintaxis dependiente del lenguaje

21.8.4 Nombres de los métodos: crear

El mensaje *crear* es la forma independiente del lenguaje UML con que se indica instanciación e inicialización. Cuando traducimos el diseño a un lenguaje de programación orientado a objetos, hay que expresarlo en términos de sus expresiones de instanciación e inicialización. Ni Java, ni C++ ni tampoco Smalltalk cuentan con un método *crear*. Por ejemplo, en C++ implica una asignación automática o una asignación en el almacenamiento libre mediante el operador *new*, seguido de una llamada al constructor. En Java implica invocar el operador *new*, seguido de la llamada a un constructor.

Por sus interpretaciones tan heterogéneas, y también por ser la inicialización una actividad extraordinariamente común, se acostumbra omitir los métodos relacionados con la creación y los constructores procedentes del diagrama de clases del diseño.

21.8.5 Nombres de los métodos: métodos de acceso

Los **métodos de acceso** son aquellos que recuperan (método accesor) o establecen (método mutador) el valor de los atributos. Una estructura común consiste en contar con un accesor y con un mutador por cada atributo y declarar privados todos los atributos (para obligar el encapsulamiento). Normalmente estos métodos no se incluyen en la descripción del diagrama de clase por la elevada razón ruido a valor que generan: para *N* atributos hay *2N* métodos sin el menor interés.

Por ejemplo, no se muestra el método *precio* de *Especificaciondeproductos* (u *obtenerPrecio*) aunque está presente, por ser *precio* un método accesor simple.

21.8.6 Nombres de los métodos: multiobjetos

Un mensaje a un multiobjeto se interpreta como si estuviera destinado al objeto contenedor/colección.

Por ejemplo, el siguiente mensaje *encontrar* dirigido a un multiobjeto ha de interpretarse como si estuviera destinado a un objeto contenedor/colección; por ejemplo, una *tabla Hashtable* (*Cálculo de Dirección*), un *map* (*mapa*) de C++ o un *Dictionary* (*Diccionario*) de Smalltalk (figura 21.6).

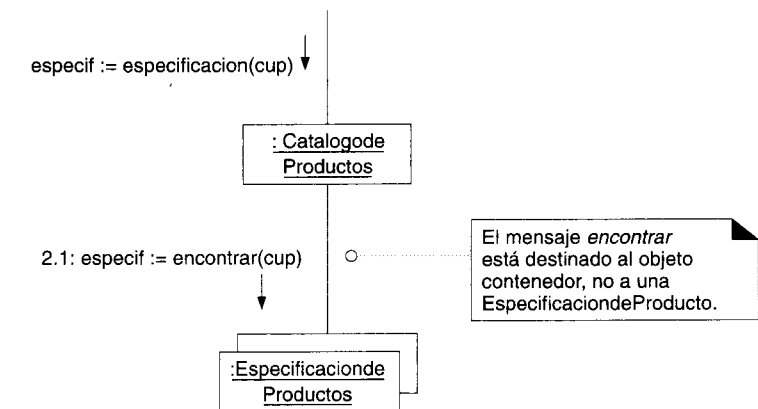


Figura 21.6 Mensaje a un multiobjeto.

Por tanto, el método *encontrar* no forma parte de la clase *EspecificaciondeProducto*, sino más bien de la definición de la clase *Hashtable* o *Dictionary*. Es, pues, incorrecto agregar *encontrar* como método a la clase *EspecificaciondeProducto*.

Estas clases contenedor/colección (como *java.util.Vector* y *java.util.Hashtable*) son clases predefinidas de las bibliotecas, y rara vez sirve mostrarlas explícitamente en el diagrama respectivo porque incorporan ruido y aportan escasa información nueva.

21.8.7 Nombres de los métodos: sintaxis dependiente del lenguaje

Algunos lenguajes, entre ellos Smalltalk, tienen una forma sintáctica para los métodos distintos al formato básico de *nombre del Metodo (lista de Parametros)*. Recomendamos utilizar el formato básico UML, aun cuando el lenguaje de la implementación planeada aplique otra sintaxis. En teoría, la traducción debería tener lugar en el momento de generar el código y no al elaborar los diagramas de clase. Pese a ello, el lenguaje UML admite otra sintaxis para especificar el método, la de Smalltalk por ejemplo.

21.8.8 Agregar más información sobre los tipos

Es opcional mostrar el tipo de los atributos, los parámetros del método y los valores de devolver método. La cuestión de si debe incluirse o no esta información debería considerarse dentro del siguiente contexto:

El diagrama de clases orientado al diseño debería elaborarse teniendo en cuenta la audiencia.

- Si vamos a crearlo en una herramienta CASE con generación automática de código, se requerirán detalles completos y exhaustivos.
- Si vamos a crearlo para que lo lean los diseñadores del software, el exceso de detalles puede influir negativamente en la razón ruido a valor.

Por ejemplo, ¿es necesario mostrar toda la información sobre los parámetros y su tipo? Respuesta: depende de cuán evidente sea la información para los destinatarios.

En la figura 21.7, el diagrama de clases del diseño y destinado al punto de venta muestra información exhaustiva sobre los tipos.

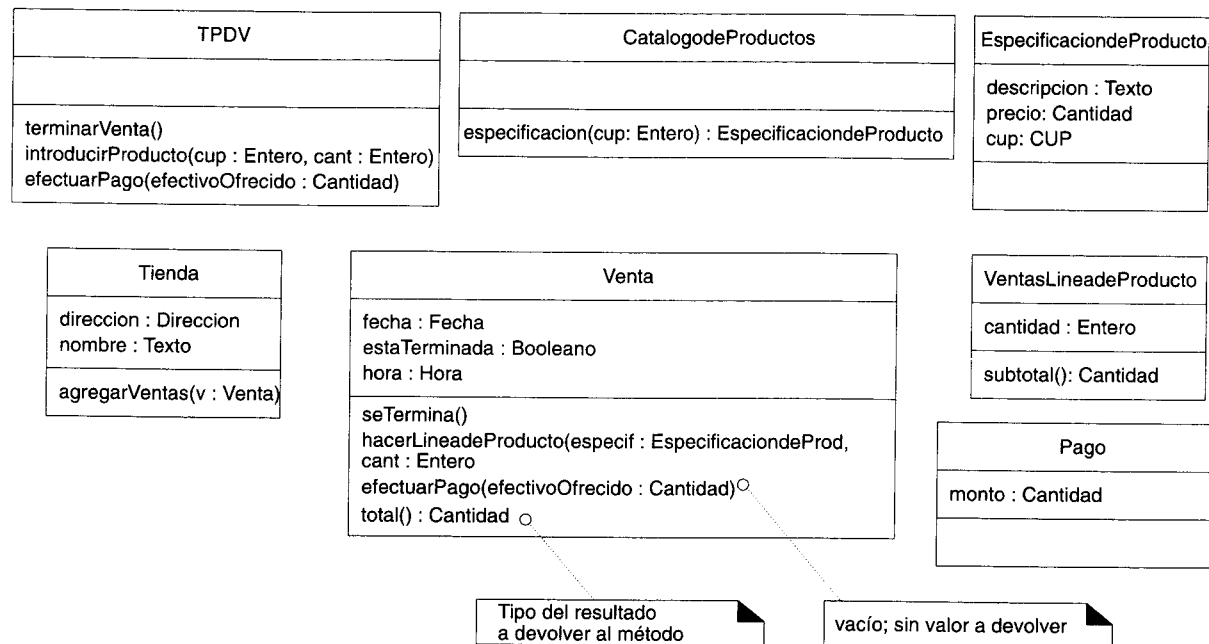


Figura 21.7 Incorporación de información sobre los tipos.

21.8.9 Incorporación de asociaciones y de navegabilidad

Se da el nombre de papel al fin de una asociación; en los diagramas de clases orientados al diseño podemos adornar el papel con una flecha de navegabilidad. La **navegabilidad** es una propiedad del papel e indica la posibilidad de navegar unidireccionalmente en una asociación, desde los objetos fuente hasta la clase destino. La navegabilidad significa visibilidad; generalmente la visibilidad de los atributos. Un ejemplo de ella se incluye en la figura 21.8.

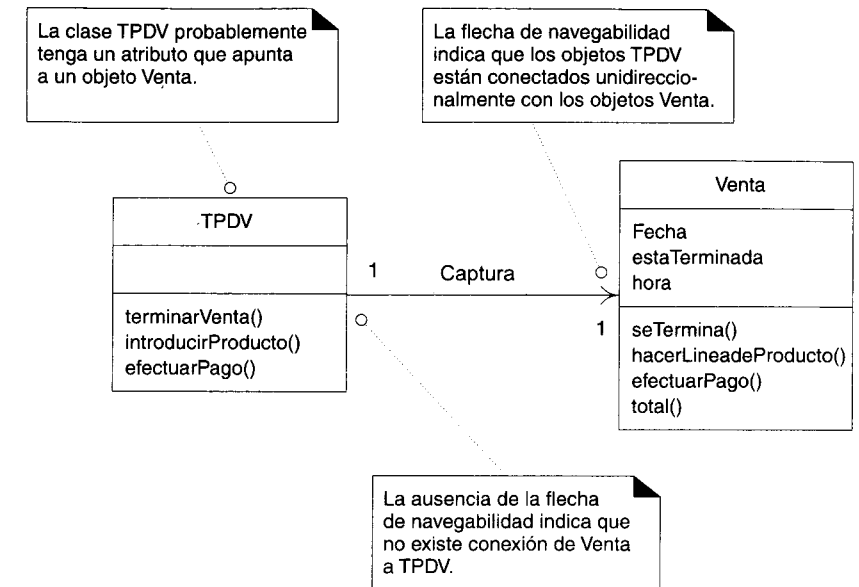


Figura 21.8 Descripción gráfica de la navegabilidad, o sea de la visibilidad de los atributos.

La interpretación usual de una asociación con una flecha de navegabilidad es la visibilidad de los atributos, desde la clase fuente hasta la clase destino. Cuando se implementa un lenguaje de programación orientado a objetos, suele traducirse como si la clase fuente tuviera un atributo que se refiere a una instancia de la clase destino. Por ejemplo, la clase **TPDV** definirá un atributo que referencia una instancia **Venta**.

En los diagramas de clase orientados al diseño, la mayoría de las asociaciones deberían completarse con las flechas necesarias de navegación.

En un diagrama de este tipo, las asociaciones se escogen con un riguroso criterio que esté orientado al software y que se rija por la necesidad de conocer: ¿cuáles asociaciones se requieren para satisfacer con los diagramas de interacción la visibilidad y las necesidades constantes de memoria? Esto contrasta con las asociaciones en el modelo conceptual, que pueden justificarse por la intención de mejorar la comprensión del dominio del problema. Una vez más captamos una distinción entre las metas de los diagramas de clase orientados al diseño y el modelo conceptual: uno es analítico y el otro una descripción de los componentes del software.

La visibilidad y las asociaciones requeridas entre las clases se indican con los diagramas de interacción. A continuación se mencionan tres situaciones comunes que revelan la necesidad de definir una asociación con una flecha de navegabilidad de A a B.

- A envía un mensaje a B.
- A crea una instancia B.
- A necesita mantener una conexión con B.

Por ejemplo, en el diagrama de colaboración de la figura 21.9 —comenzando con el mensaje *crear* dirigido a una *Tienda* y a partir del contexto más amplio de otros diagramas de este tipo—, podemos observar que la *Tienda* probablemente tenga una conexión permanente con las instancias que genera. También es razonable que *CatalogodeProductos* necesite una conexión permanente con la colección de *EspecificacionesdeProducto* que genere. De hecho, el creador de otro objeto suele requerir ese tipo de conexión con él.

Por tanto, las conexiones en cuestión estarán presentes como asociaciones en el diagrama de clases.

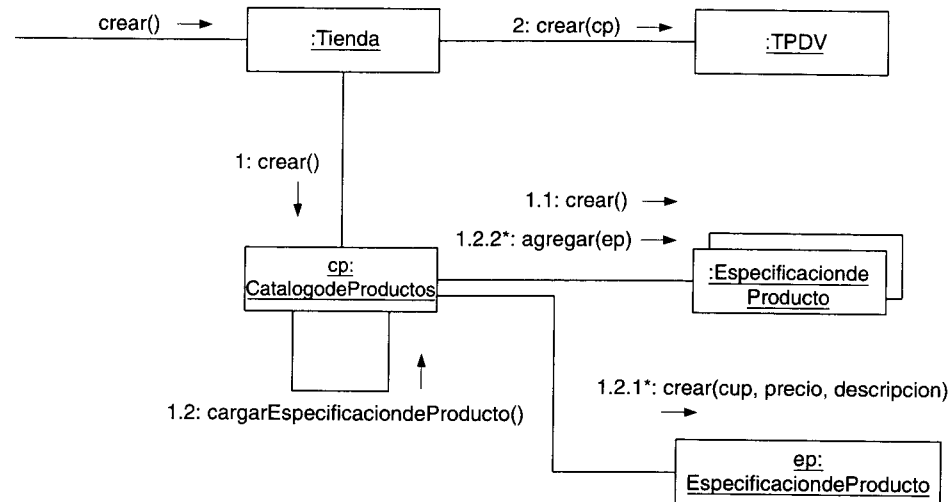


Figura 21.9 La navegabilidad se identifica a partir de los diagramas de colaboración.

Basándose en el criterio anterior de asociaciones y de navegabilidad, el análisis de los diagramas de colaboración obtenidos para la aplicación de punto de venta producirá un diagrama de clases (figura 21.10) con las siguientes asociaciones. (Por razones de claridad, se oculta la información exhaustiva sobre los tipos.)

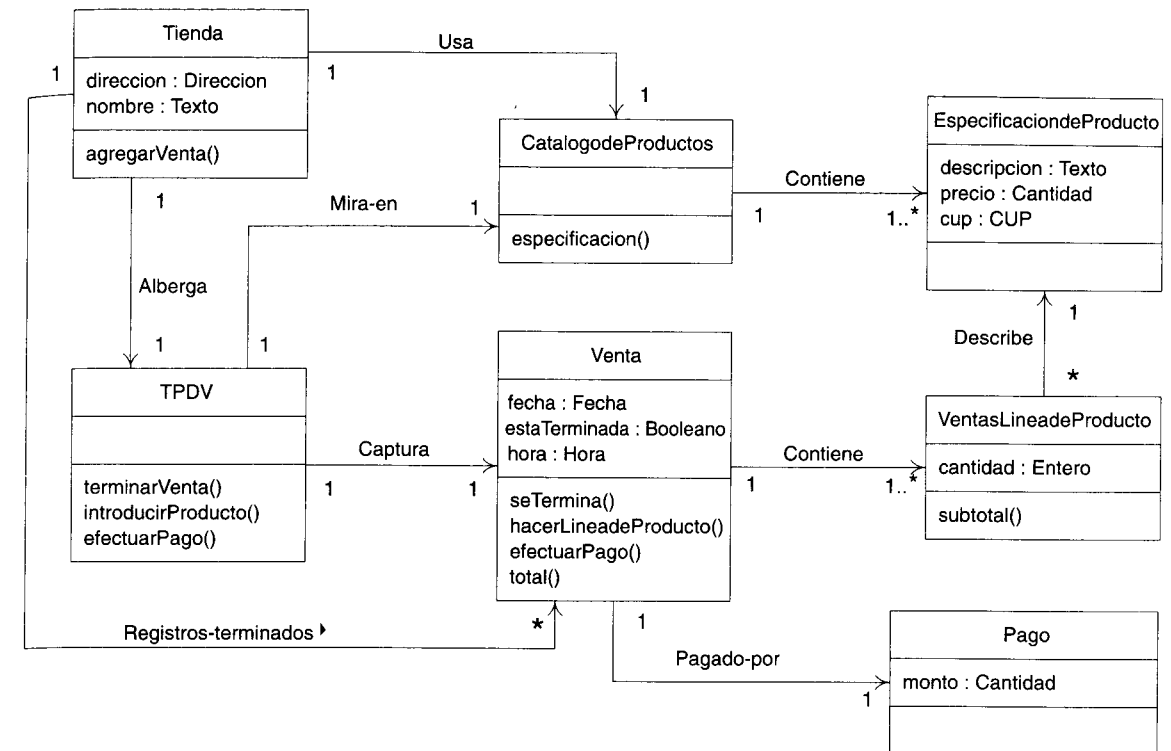


Figura 21.10 Asociaciones con símbolos de navegabilidad.

Nótese que éste no es exactamente el mismo conjunto de asociaciones que el generado para los diagramas de clases en el modelo de investigación. Por ejemplo, no existía la asociación *Mira-en* entre *TPDV* y *CatalogodeProductos* en el modelo conceptual: en ese momento no se descubrió que fuera una relación importante ni duradera. Pero durante la preparación de los diagramas de colaboración se decidió que el objeto de software *TPDV* debería tener una conexión permanente con un software *CatalogodeProductos* a fin de consultar *EspecificacionesdeProducto*.

21.8.10 Agregación de las relaciones de dependencia

El lenguaje UML incluye una **relación general de dependencia**, la cual indica que un elemento (de cualquier tipo como clases, casos de uso y otros) conoce la existencia de otro. Se denota con una línea punteada y con flecha. En el diagrama de clases, la relación de dependencia sirve para describir la visibilidad entre atributos que no se relaciona con ellos; en otras palabras, la visibilidad del parámetro global o declarada localmente. En cambio, la visibilidad simple de atributos se indica con una flecha normal de asociación y con una flecha de navegabilidad.

Por ejemplo, el objeto de software *TPDV* recibe un objeto devuelto del tipo *EspecificaciondeProducto* proveniente del mensaje de especificación que envió a un *CatalogodeProductos*. Así, *TPDV* tiene una visibilidad de corto plazo localmente declarada en *EspecificacionesdeProducto*. Y *Venta* recibe una *EspecificaciondeProducto* como parámetro en el mensaje *hacerLineadeProducto* tiene la visibilidad del parámetro en uno. Estas visibilidades no relacionadas con atributos pueden denotarse con la línea punteada y con flecha para indicar una relación de dependencia (figura 21.11). (La curvatura de las líneas de dependencia carece de importancia; en este ejemplo se usó por razones gráficas.)

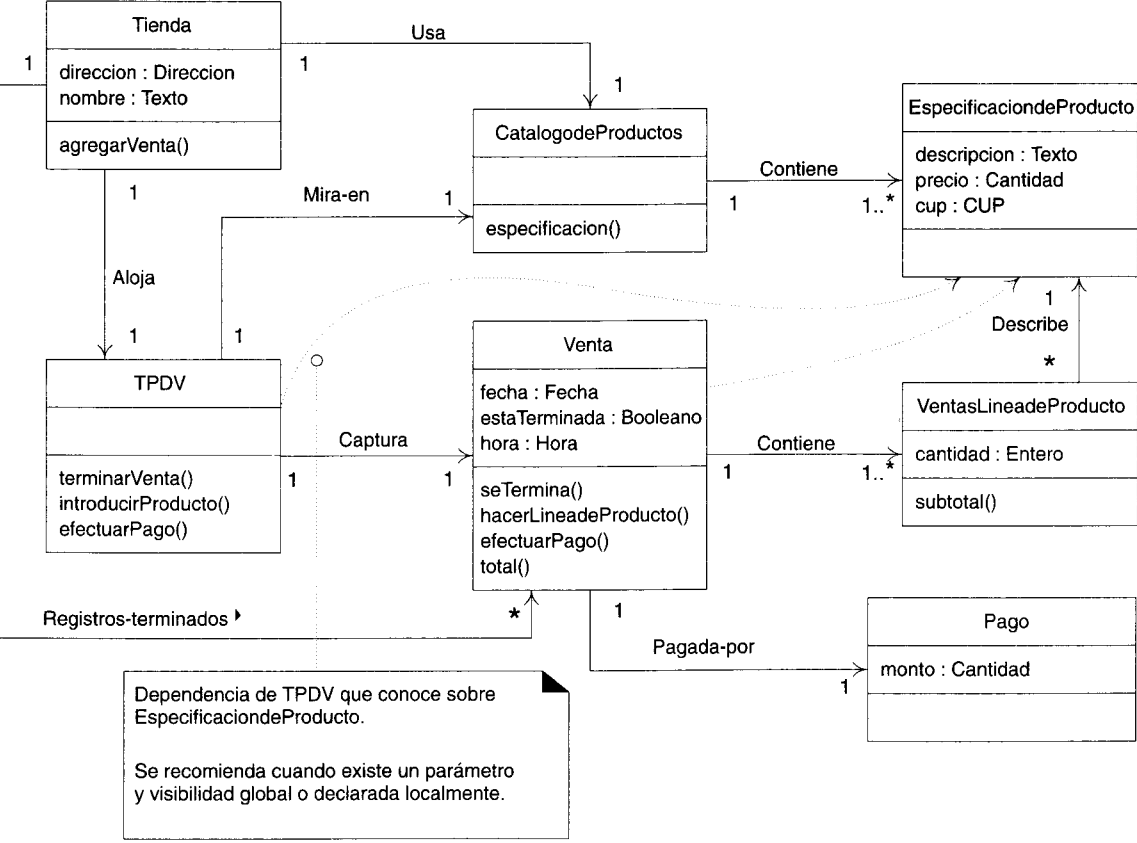


Figura 21.11 Relaciones de dependencia que indican una visibilidad no relacionada con atributos.

21.9 Notación de los detalles de los miembros

El lenguaje UML ofrece una rica notación para describir las características de los miembros de la clase e interfaz; por ejemplo, la visibilidad y los valores iniciales. Se supone que los atributos son privados por omisión. Un ejemplo de ello se da en la figura 21.12.

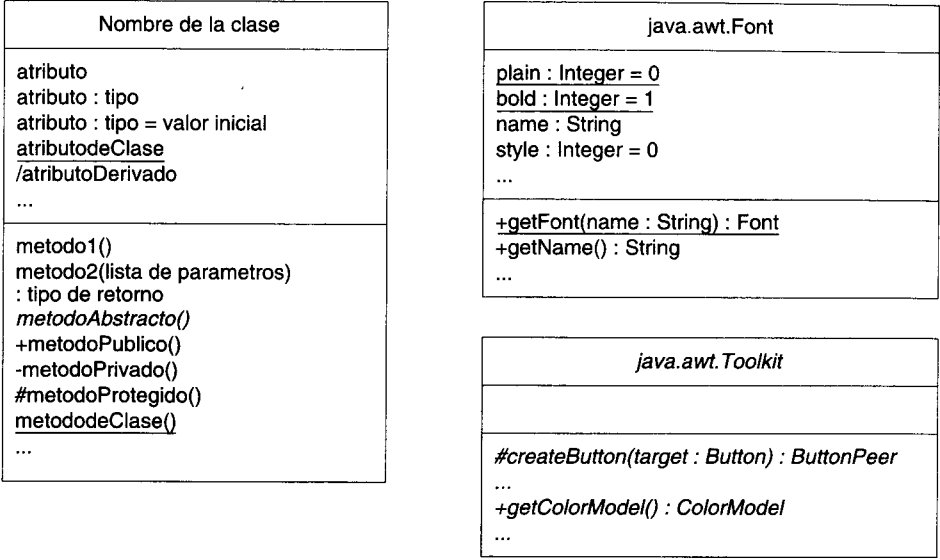


Figura 21.12 Notación usada para los detalles de los miembros de la clase.

La iteración actual del diagrama de clases del diseño para el punto de venta (figura 21.13) no abarca muchos detalles interesantes referentes a los miembros; todos los atributos son privados y públicos todos los métodos.

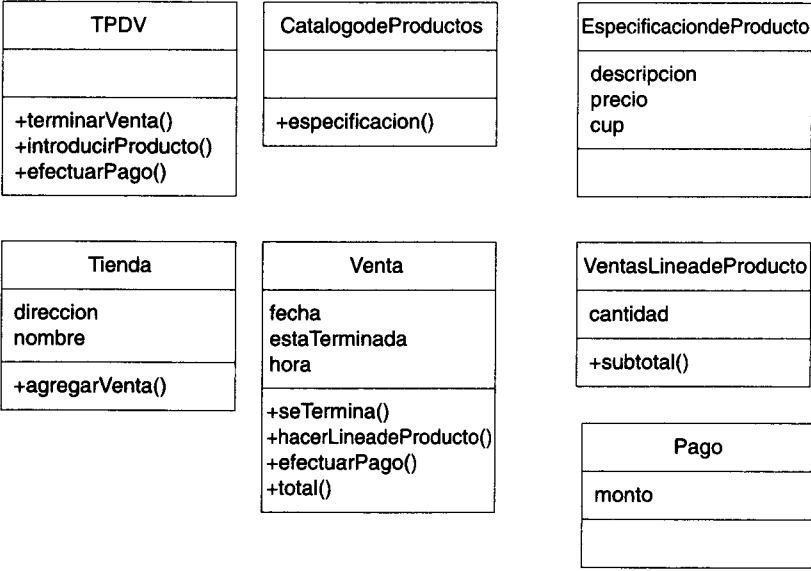


Figura 21.13 Detalles de los miembros en el diagrama de clases para el punto de venta.

21.10 Modelos muestra

Los diagramas de clase del diseño constituyen el modelo de clases, o sea el de las clases de software y las interfaces.

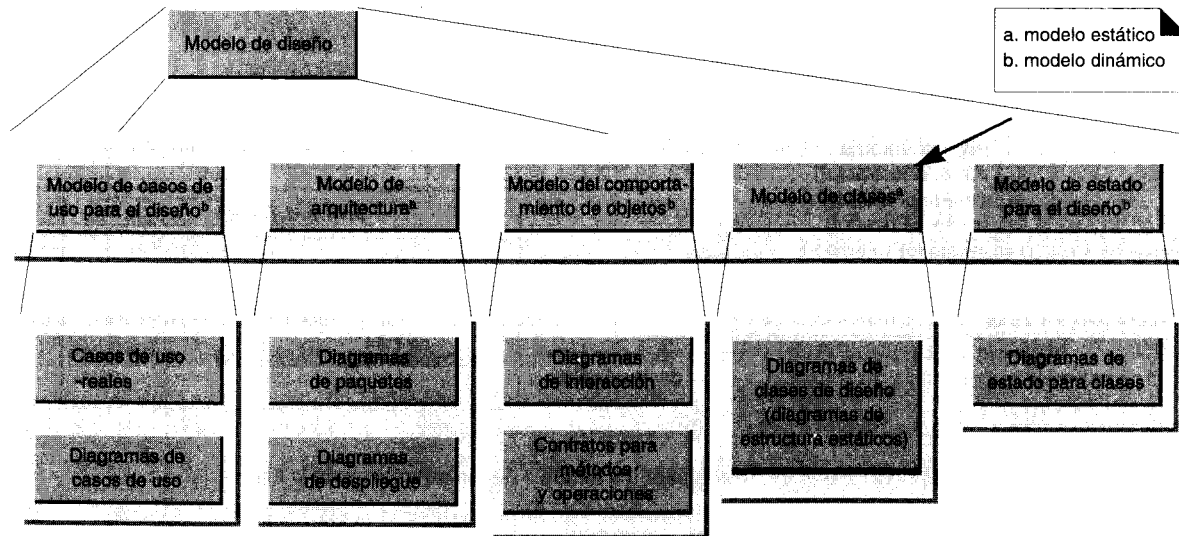


Figura 21.14 El modelo de diseño.

21.11 Resumen

Para la aplicación de la terminal situada en el punto de venta hemos preparado un diagrama de clases del diseño como parte del modelo del diseño que aparece en la figura 21.14. Ilustra la definición de las clases como componentes de software. Se trata de un tipo de diagramas sucintos y muy informativos, cualidades convenientes en la notación. Debido a tales cualidades, cuando se trabaja con un sistema orientado a objetos ya existente o cuando se diseña un nuevo sistema, los desarrolladores tienden a recurrir más a estos diagramas que a cualquier otro.

ALGUNOS ASPECTOS DEL DISEÑO DE SISTEMAS

Objetivos

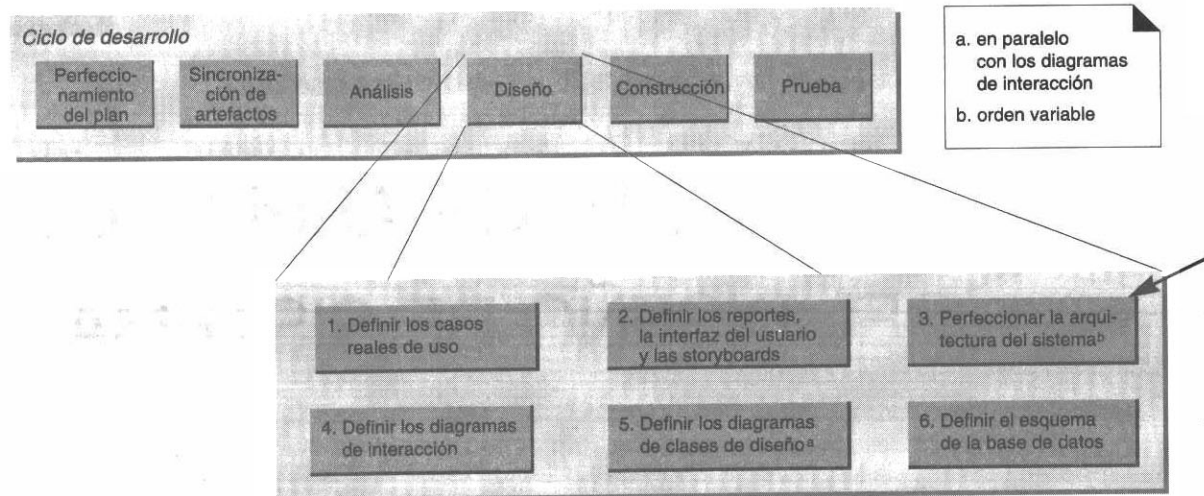
- Diseñar la arquitectura de un sistema a partir de las capas y de las particiones.
- Describir gráficamente el diseño arquitectónico mediante los diagramas de paquetes del UML.
- Aplicar los patrones Fachada, Separación de Modelo-Vista y Publicar-Suscribir, así como la notificación de eventos, para dar soporte a varias metas arquitectónicas.

22.1 Introducción

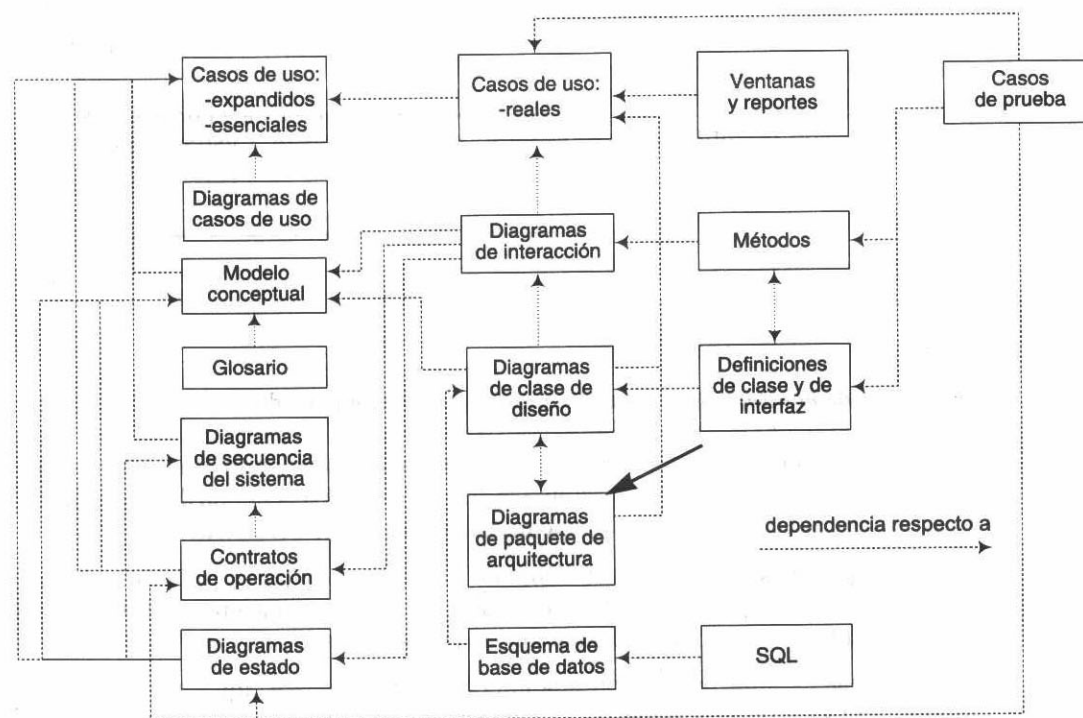
En el estudio anterior de casos nos centramos en los objetos del dominio del problema (entre ellos, *Venta*), porque así se definen los conceptos y el comportamiento básicos de un sistema. Pero un sistema se compone de muchos subsistemas, uno de los cuales son los objetos del dominio. Un sistema ordinario de información ha de conectarse a la interfaz del usuario y a un mecanismo de almacenamiento persistente.

En este capítulo examinaremos brevemente la arquitectura más amplia de un sistema, la comunicación y el acoplamiento entre subsistemas y la notación del UML —diagramas de paquetes— que nos permiten describir los subsistemas.

Los sistemas de información con una interfaz para el usuario (que entre otras cosas incluye software de aplicación como los procesadores de palabras) son la categoría más común de aplicación; de ahí que comencemos analizando las cuestiones más usuales de la arquitectura que se relacionan con ellos. Muchos de los conceptos aquí expuestos podrán utilizarse con otros tipos de aplicaciones.



Actividades de la fase de diseño dentro de un ciclo de desarrollo.



Dependencias de los artefactos durante la fase de construcción.

22.2 Arquitectura clásica de tres capas

Una arquitectura común de los sistemas de información que abarca una interfaz para el usuario y el almacenamiento persistente de datos se conoce con el nombre de **arquitectura de tres capas** [Gartner95]; podemos observarla en la figura 22.1. He aquí una descripción clásica de las capas verticales:

1. **Presentación:** ventanas, reportes, etcétera.
2. **Lógica de aplicaciones:** tareas y reglas que rigen el proceso.
3. **Almacenamiento:** mecanismo de almacenamiento persistente.

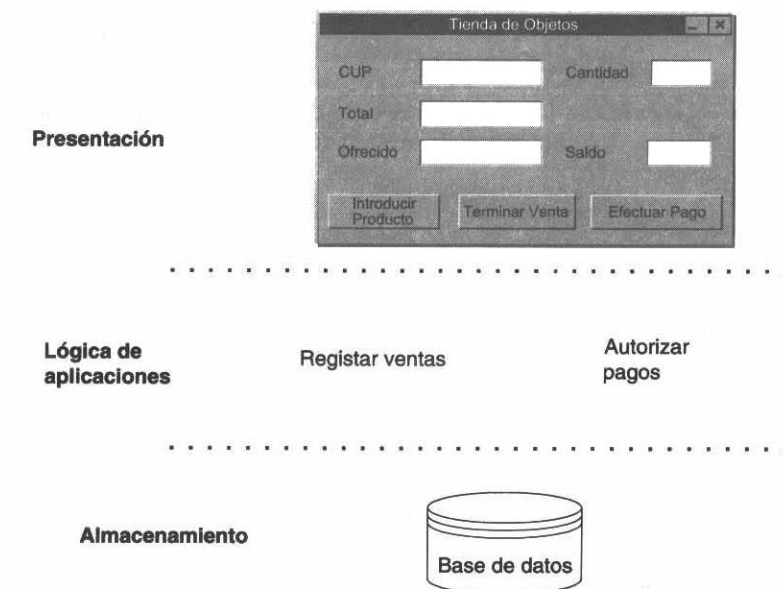


Figura 22.1 Vista clásica de una arquitectura de tres capas.

La calidad tan especial de la arquitectura de tres capas consiste en aislar la lógica de la aplicación y en convertirla en una capa intermedia bien definida y lógica del software. En la capa de presentación se realiza relativamente poco procesamiento de la aplicación; las ventanas envían a la capa intermedia peticiones de trabajo. Y éste se comunica con la capa de almacenamiento del extremo posterior.

Esta arquitectura contrasta con el diseño de **dos capas**, donde —por ejemplo— colocamos la lógica de aplicaciones dentro de las definiciones de ventana, que leen y escriben directamente en una base de datos; no hay una capa intermedia que separe la lógica. Una de sus desventajas es la imposibilidad de representar la lógica en componentes aislados, lo cual impide reutilizar el software. No es posible distribuir la lógica de aplicaciones en una computadora diferente.

22.3 Arquitecturas multicapas orientadas a objetos

Una arquitectura multicapas que se adecue a los sistemas de información orientados a objetos incluye la división de las responsabilidades que encontramos en la arquitectura clásica de tres capas. Las responsabilidades se asignan a los objetos de software.

22.3.1 Descomposición de la capa de la lógica de las aplicaciones

En un diseño orientado a objetos, la capa de la lógica de aplicaciones se divide en otras menos densas. Esta arquitectura, organizada a partir de las clases de software, se muestra en la figura 22.2. La capa de la lógica de aplicaciones está constituida por las siguientes capas:

- **Objetos del dominio:** clases que representan los conceptos del dominio; por ejemplo, una venta.
- **Servicios:** los objetos servicio de las funciones como la interacción de bases de datos, los reportes, las comunicaciones y la seguridad.

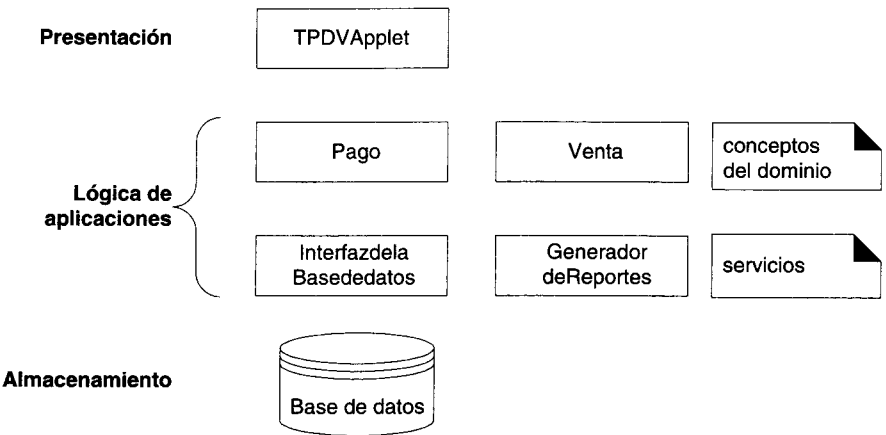


Figura 22.2 Descomposición de la capa de lógica de aplicaciones en estratos o subcapas más detalladas.

22.3.2 Más allá de las tres capas: arquitectura multicapas

Cuando vemos la arquitectura desde la perspectiva de una descomposición más detallada que se observa en la figura 22.2, podemos prescindir de la designación “arquitectura de tres capas” y hablar en cambio de **arquitecturas multicapas**; en ellas está implícita la capa intermedia de la lógica de aplicaciones.

Es posible agregar más capas y descomponer ulteriormente las ya existentes. Así, la capa *Servicios* puede dividirse en servicios de alto y bajo nivel (por ejemplo, generación de reportes frente a entrada/salida).

22.3.3 Despliegue

Una arquitectura *lógica* de tres capas puede desplegarse o implementarse *físicamente* en varias configuraciones, entre las cuales se encuentran:

1. Capas de la lógica de presentación y de aplicaciones en la computadora del cliente, en su almacenamiento o en su servidor.
2. La presentación en la computadora del cliente, la lógica de aplicaciones en un servidor de la aplicación y el almacenamiento en un servidor de datos independiente.

Dado el mayor uso de los lenguajes y tecnologías que soportan fácilmente el cómputo distribuido, Java entre otros, también el despliegue de los subsistemas se irá realizando de manera distribuida cada vez más.

22.3.4 Motivos para utilizar la arquitectura multicapas

Entre los motivos por los cuales se recurre a la arquitectura multicapas se cuentan los siguientes:

- Aislamiento de la lógica de aplicaciones en componentes independientes susceptibles de reutilizarse después en otros sistemas.
- Distribución de las capas en varios nodos físicos de cómputo y en varios procesos. Esto puede mejorar el desempeño, la coordinación y el compartir la información en un sistema de cliente-servidor.
- Asignación de los diseñadores para que construyan determinadas capas; por ejemplo, un equipo que trabaje exclusivamente en la capa de presentación. Y así se brinda soporte a los conocimientos especializados en las habilidades de desarrollo y también a la capacidad de realizar actividades simultáneas en equipo.

22.4 Cómo mostrar la arquitectura con paquetes de UML

El lenguaje UML ofrece el mecanismo **paquete** que permite describir los grupos de elementos o subsistemas. Un paquete es un conjunto de cualquier tipo de elementos de un modelo: clases, casos de uso, diagramas de colaboración u otros paquetes (los anidados). Un sistema puede examinarse íntegramente dentro del ámbito de un solo paquete de alto nivel: el paquete *Sistema*. El paquete define un espacio de un nombre anidado, de modo que los elementos del mismo nombre pueden duplicarse dentro de varios paquetes.

22.4.1 Notación de los paquetes de UML

Un paquete se muestra gráficamente como una carpeta con etiquetas (figura 22.3). Los paquetes subordinados se incluyen en su interior. El nombre del paquete se encuentra dentro de la etiqueta si el paquete describe sus elementos; en caso contrario estará en el centro de la misma carpeta.

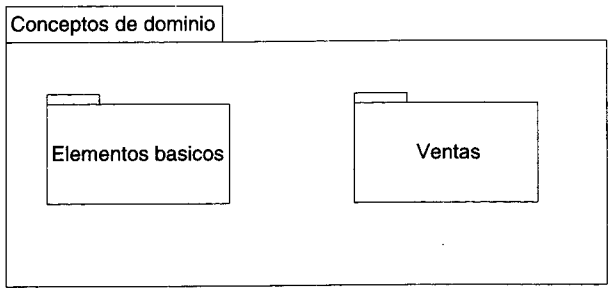


Figura 22.3 Paquetes en el lenguaje UML.

22.4.2 Diagramas de los paquetes descriptivos de la arquitectura

Los paquetes nos permiten describir la arquitectura de un sistema con la notación UML. En la figura 22.4 se incluye una vista alterna de la figura 22.2, que representa los agrupamientos lógicos mediante los diagramas de un paquete de UML. A este tipo de diagrama podemos llamarlo **diagrama de paquetes de la arquitectura**.

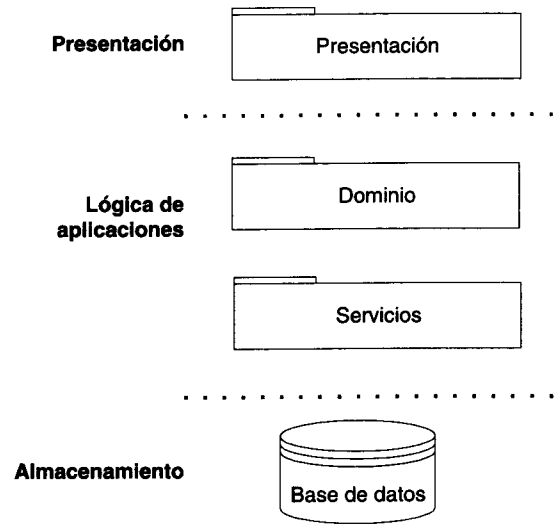


Figura 22.4 Unidades arquitectónicas expresadas en términos de los paquetes de UML.

22.4.3 Ejemplo de paquetes y de dependencias

La figura 22.5 contiene una descomposición más detallada de algunos paquetes comunes en la arquitectura de un sistema de información, así como las dependencias entre ellos.

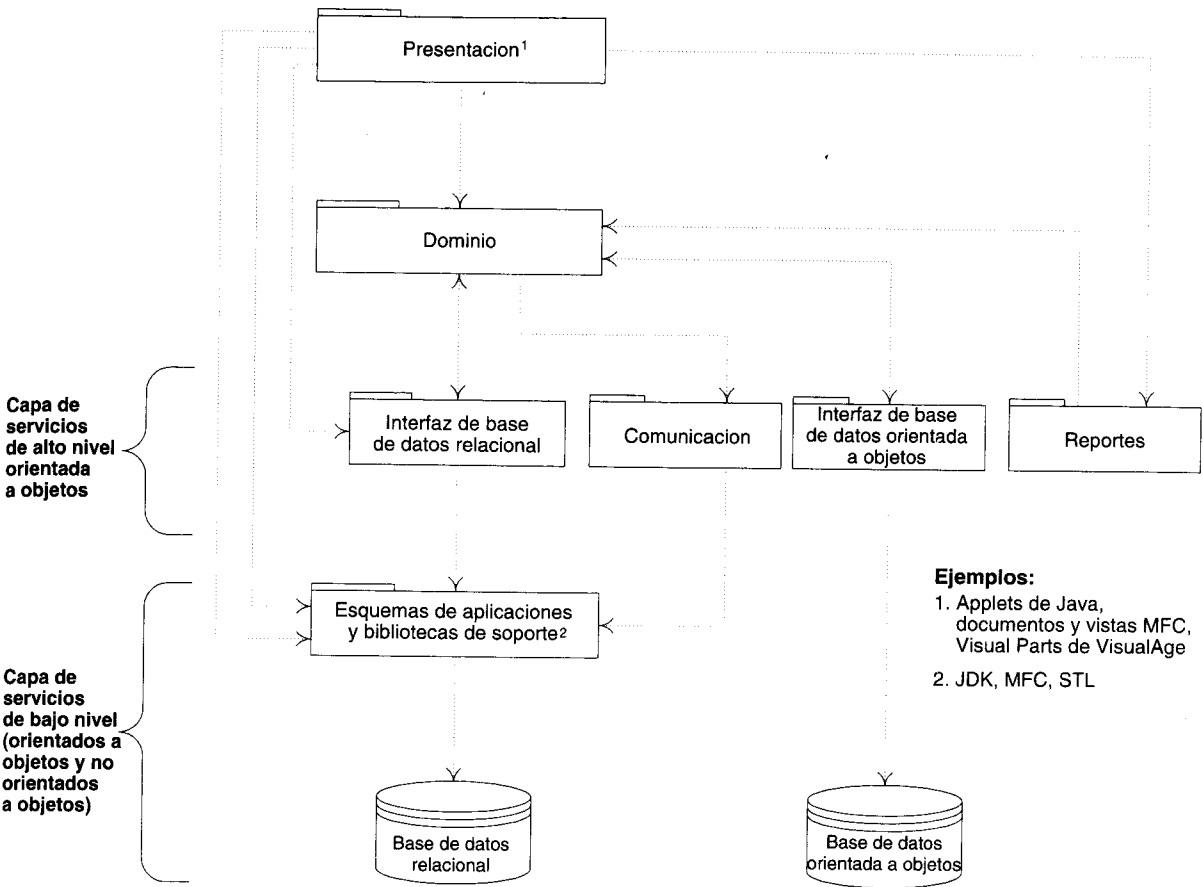


Figura 22.5 Arquitectura detallada.

Comentarios sobre los paquetes

- Los paquetes de la interfaz de bases de datos relacionales y de las orientadas a objetos ofrecen mecanismos para comunicarse con ellas. El proveedor suministrará una interfaz de base de datos orientada a objetos. Pero es necesario adaptar una interfaz de base de datos relacional o bien comprarle a otro proveedor un producto que desempeñe esa función.
- Entre los paquetes de servicios de alto nivel orientados a objetos podemos citar los siguientes: reportes, interfaces de bases de datos, seguridad y comunicaciones entre procesos preparados todos ellos con la tecnología de objetos. Normalmente son los diseñadores de aplicaciones quienes los escriben. En cambio, los servicios de bajo nivel ofrecen las funciones básicas, como la manipulación de ventanas y de archivos, y suelen hacerlo como bibliotecas estándar de lenguajes o adquirirse de otro proveedor.

- Las bibliotecas de soporte y de esquemas de aplicaciones casi siempre brindan soporte para crear ventanas, para definir los coordinadores de aplicaciones, acceder a bases de datos y a archivos, para la comunicación entre procesos, etcétera.

Comentarios sobre la dependencia

- Las relaciones de dependencia (línea punteada con flecha) indican si el paquete conoce el acoplamiento con otro. La *ausencia* de dependencia del paquete A con el B significa que los componentes de A *no* tienen referencias a ninguna clase, componentes, interfaz, método o servicios de B.
- Nótese que el paquete de dominio no tiene dependencia (acoplamiento) con las capas de presentación. Esto ejemplifica el principio de *Separación de Modelo-Vista* que comentaremos más adelante.

22.5 Identificación de los paquetes

Agrupe los elementos en un paquete aplicando la siguiente directriz:

Agrupe los elementos para ofrecer en un paquete un servicio común (o una familia de servicios relacionados), con un nivel relativamente alto de acoplamiento y colaboración.

En cierto nivel de abstracción, se verá el paquete como muy cohesivo (tiene responsabilidades estrechamente relacionadas).

En cambio, habrá relativamente bajo acoplamiento y colaboración entre los elementos de los paquetes.

22.6 Estratos y particiones

Podemos caracterizar una arquitectura multicapas como compuesta de estratos (subcapas) y particiones (figura 22.6).

Los **estratos** de una arquitectura representan las capas verticales, mientras que las **particiones** representan la división horizontal de subsistemas relativamente paralelos de un estrato. Por ejemplo, el estrato *Servicios* puede dividirse en particiones como *Seguridad* y *Reportes*.

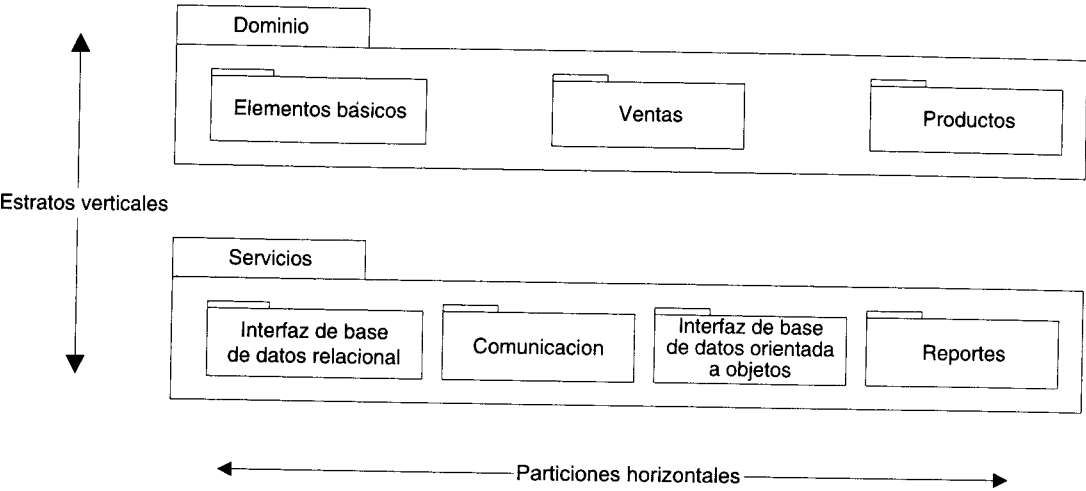


Figura 22.6 Estratos (subcapas) y particiones.

22.6.1 Arquitectura multicapas “laxa”

En las arquitecturas multicapas orientadas a objetos, los estratos *no* están acopladas en un sentido tan limitado como un protocolo de redes que se base en el modelo OSI de 7 capas. El modelo de protocolo comprende la restricción de que los elementos de la capa N sólo podrán acceder a los servicios de la capa o estrato inmediatamente inferior N-1. Aunque tal limitación es posible, la arquitectura suele ser de “capas laxas” o de “capas transparentes” [BMRSS96], donde los elementos de una capa o estrato se comunican con otras. La figura 22.5 presenta un acoplamiento ordinario entre estratos y particiones. Por ejemplo, el estrato de presentación (o de la coordinación de aplicaciones) normalmente está acoplada al de dominio y servicios.

22.7 Visibilidad entre las clases de paquetes

- ¿Cómo deberían relacionarse entre sí los componentes de un paquete? La visibilidad respecto a las clases en varios paquetes suele conformar el siguiente patrón:
- **Acceso a los paquetes del dominio:** otros paquetes, generalmente el de presentación, tienen acceso a la visibilidad de *muchas* clases que representan los conceptos del dominio.
 - **Acceso a los paquetes de servicios:** otros paquetes, generalmente los de dominio y de presentación, tienen acceso a la visibilidad sólo respecto a *una* clase o unas cuantas en determinado paquete de servicios (casi siempre una clase **Fachada** como veremos luego).

- **Acceso a los paquetes de presentación:** ningún otro paquete tendrá acceso directo a la visibilidad de la capa de presentación. La comunicación es indirecta (mediante el patrón **Observador**, como veremos luego), si es que existe.

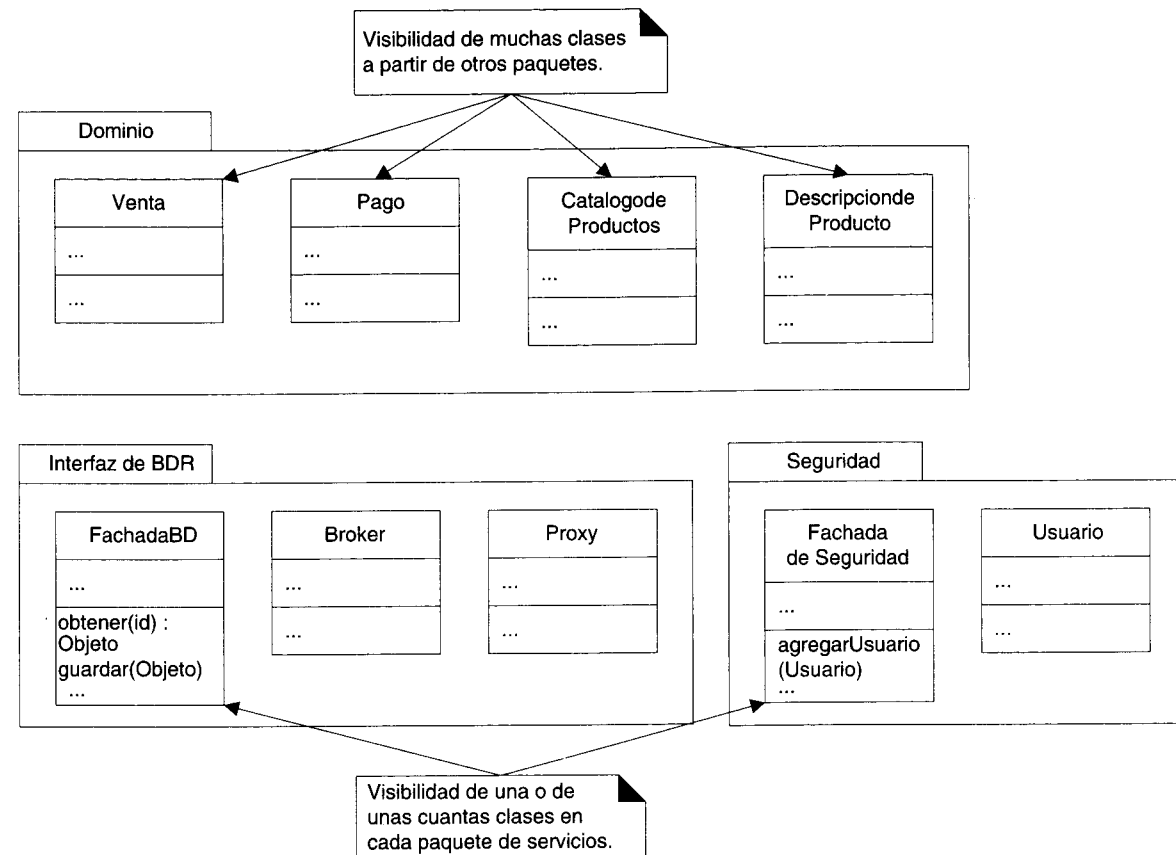
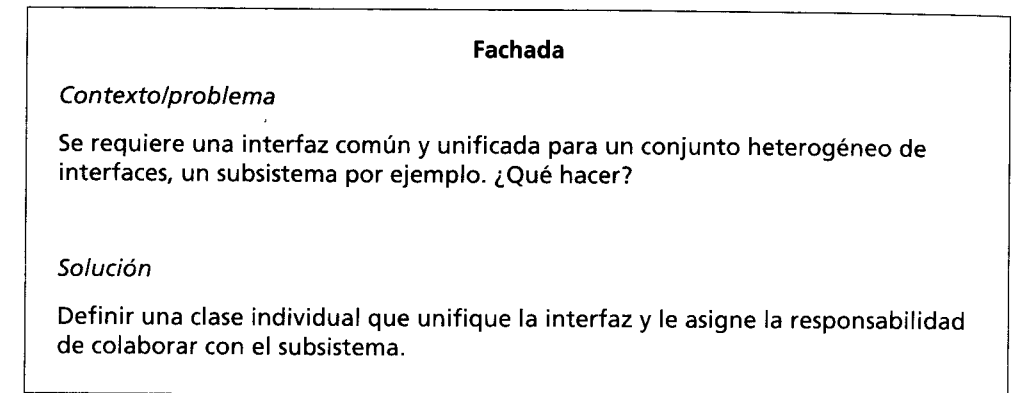


Figura 22.7 Visibilidad entre paquetes.

22.8 Interfaz de los paquetes de servicios: el patrón Fachada

¿Cómo deberían los componentes coordinarse con las clases de un paquete de servicios —por ejemplo, el paquete *Interfaz de Base de Datos Relacional*, que ofrece servicios para realizar transformaciones entre los objetos y los renglones de las tablas? Cuando se define una clase con una interfaz común a un grupo de componentes o a un conjunto heterogéneo de interfaces, a esa clase le damos el nombre de **Fachada** [GHJV95]. Los elementos heterogéneos pueden ser las clases de un paquete, un conjunto de funciones, un esquema o un subsistema (local o remoto).



El patrón Fachada a menudo sirve para suministrar una interfaz pública a un paquete de servicios. Por ejemplo, si se dispone de un paquete de *Interfaz de Base de Datos Relacional* con muchas clases internas, podemos definir una clase como *FachadaBD* que ofrezca la interfaz pública común con los servicios del paquete (figura 22.7). Las clases de otros paquetes envían mensajes únicamente a una instancia de ella y no se acoplan a otras clases del paquete. *FachadaBD* colabora con otras clases (privadas) del paquete para prestar servicios. Este diseño repercute en un bajo acoplamiento.

22.9 Sin visibilidad directa respecto a las ventanas: el patrón de Separación Modelo-Vista

¿Qué tipo de visibilidad deberían tener otros paquetes respecto a la capa de presentación? ¿Cómo deberían comunicarse con las ventanas las clases no relacionadas con ellas? Por lo regular conviene que no haya un acoplamiento directo de otros componentes con los objetos ventana porque éstas se encuentran relacionadas con una aplicación en particular, mientras que (en teoría) podemos reutilizar en nuevas aplicaciones los componentes sin acceso a ellas o unirlos a una nueva interfaz. El principio que se aplica en este caso es el patrón Separación Modelo-Vista.

Dentro de este contexto, **modelo** es sinónimo de capa del dominio de objetos. **Vista** es sinónimo de objetos presentación, como ventanas, applets y reportes.

El patrón de **Separación Modelo-Vista**¹ (conocido también con el nombre de **Separación Dominio-Presentación**) establece que los objetos modelo (dominio) no deberían conocer *directamente* los objetos vista (presentación) ni estar directamente acoplados a ellos.

¹ Se describe totalmente como un patrón en [BMRSS96] con el nombre de *Modelo-Vista-Controlador*. En realidad, la parte del patrón correspondiente a *Controlador* (manejador de entrada de bajo nivel) es un anacronismo, ya que las responsabilidades del controlador suelen quedar incluidas dentro de las vistas y ser proporcionadas por el sistema operativo; los controladores se incluyeron hace años en Smalltalk-80 antes del advenimiento de los modernos sistemas operativos y los servicios gráficos de las ventanas. El patrón "Separación de Modelo-Vista" describe mejor el propósito y la arquitectura moderna.

Los métodos de una clase de modelo no deberían contener instrucciones que envíen mensajes directos a un objeto vista; este tipo de clase no debería conocer las interfaces del usuario ni relacionarse con ellas mediante códigos.

Una clase de modelos no tendrá visibilidad directa respecto a una clase vista. Pero, como veremos, se admite la visibilidad indirecta.

Una recomendación más de este patrón es que las clases del dominio (vista) encapsulen la información y el comportamiento relacionados con la lógica de aplicaciones.

Las clases de ventanas (vistas) son relativamente sencillas; se encargan de la entrada y de la salida, sin que conserven los datos ni ofrezcan directamente la funcionalidad de las aplicaciones.

Separación Modelo-Vista

Contexto/problema

Conviene desacoplar los objetos dominio (modelo) y las ventanas (vistas), a fin de brindar soporte a un mayor reuso de los objetos dominio y reducir al mínimo el impacto que los cambios de la interfaz tienen en ellos. ¿Qué hacer?

Solución

Definir las clases de dominio (modelo) para que no tengan acoplamiento ni visibilidad directa respecto a las clases ventana (vista) y para que los datos de la aplicación y de la funcionalidad se conserven en las clases de dominio, no en las de ventana.

Supongamos, por ejemplo, que (en Java) existe un objeto *TPDVApplet* que muestre información proveniente de un objeto *Venta* y que envía mensajes de eventos de sistema a un objeto *TPDV*.

El objeto *TPDVApplet* puede tener visibilidad respecto a los objetos dominio y enviar los mensajes; por ejemplo un objeto *TPDV* (el controlador de los eventos del sistema) y *Venta*.

Pero los objetos *TPDV* y *Venta* no conocerán la existencia de la ventana; ninguno de ellos contiene afirmaciones relacionadas con ventanas ni con la lógica de presentación.

Por tanto, no conviene que *TPDV* envíe un mensaje *presentarMensaje* a la ventana, como se observa en la figura 22.8.

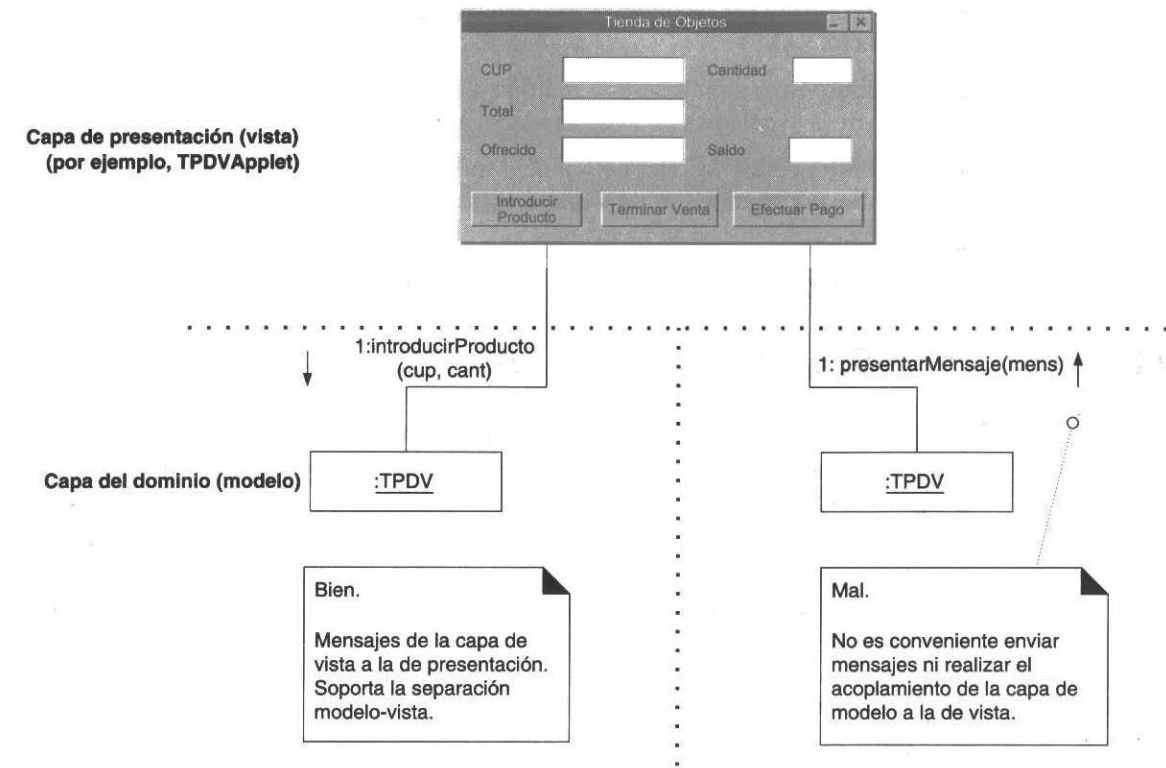


Figura 22.8 Comparación entre la conformidad y la violación del patrón Separación Modelo-Vista.

En síntesis, los objetos de la capa de dominio no tienen comunicación directa con la de presentación.

En cambio, es aceptable que las vistas tengan visibilidad respecto a los modelos; por ejemplo, un objeto *TPDVApplet* puede enviar mensajes al objeto *TPDV*.

22.9.1 Motivos del patrón de Separación Modelo-Vista

Entre las razones por las cuales se emplea el patrón de separación Modelo-Vista, podemos citar las siguientes:

- Dar soporte a las definiciones de modelos cohesivos que se centran en los procesos de dominio más que en las interfaces de computadora.
- Permitir desarrollar independientemente el modelo y las capas de interfaz para el usuario.
- Reducir al mínimo el impacto que los cambios de requerimientos de la interfaz tienen en la capa de dominio.

- Permitir conectar fácilmente otras vistas a la capa actual de dominio, sin que esto la afecte.
- Permitir vistas simultáneas y múltiples del mismo objeto modelo; por ejemplo, la vista de la estadística de ventas en un diagrama tabular.
- Permitir ejecutar la capa del modelo independiente de la capa de interfaz para el usuario; por ejemplo, en un sistema de procesamiento de mensajes o de modo lotes.
- Permitir transportar fácilmente la capa de modelo a otro esquema de interfaz para el usuario.

22.9.2 El patrón de Separación Modelo-Vista y la comunicación indirecta

¿Cómo obtienen las ventanas información para mostrarla? Generalmente basta que envíen mensajes a los objetos de dominio, solicitando la información que después despliegan en dispositivos: un modelo de **escrutinio** o de **jalar desde arriba** de las actualizaciones de la presentación visual.

No obstante, en ocasiones un modelo de escrutinio resulta insuficiente: los objetos de dominio necesitan comunicarse (indirectamente) con las ventanas para lograr una actualización permanente y de tiempo real de la presentación visual, a medida que vaya cambiando el estado de la información de los objetos del dominio. En tal caso suelen ocurrir las siguientes situaciones:

- Aplicaciones del monitoreo; por ejemplo, administración de las redes de telecomunicaciones.
- Aplicaciones de simulación que requieren visualización; por ejemplo, modelos aerodinámicos.

En las situaciones anteriores, se requiere un modelo de **empujar desde abajo** de la actualización de la presentación visual. Dada la restricción del patrón de Separación Modelo-Vista, esto hace necesaria la comunicación *indirecta* entre otros objetos y ventanas: empujar hacia arriba la notificación para actualizar desde abajo, aunque en forma indirecta.

22.10 La comunicación indirecta en un sistema

El patrón de Separación Modelo-Vista es uno de muchos ejemplos en que debe darse una comunicación indirecta entre los elementos del sistema. En los lenguajes comunes orientados a objetos, un mensaje entre un objeto emisor y un objeto receptor exige que el emisor tenga visibilidad directa del receptor. En consecuencia, hacen falta otros mecanismos además del envío directo de mensajes, si conviene desacoplar al emisor y al receptor o si se necesita brindar soporte a la transmisión de una “señal”.

En esta sección examinaremos las variaciones de la comunicación indirecta y la manera en que dan soporte a las metas arquitectónicas, entre ellas el bajo acoplamiento y la Separación Modelo-Vista.

22.10.1 El patrón Publicar-Suscribir

La solución de la comunicación indirecta que se requiere de los objetos de dominio a las ventanas es el patrón **Observador**, llamado también **Publicar-Suscribir**. Este método se explica en [GHJV95] con el nombre de **Observador** y en [BMRSS96] como **Editor-Suscriptor**.

Publicar-Suscribir

Contexto/problema

Un cambio de estado (un evento) ocurre dentro de un Editor del evento y otros objetos dependen de él o están interesados en él (suscriptores del evento). Pero el Editor no debería tener conocimiento directo de sus Suscriptores. ¿Qué hacer?

Solución

Defina un sistema de notificación de eventos para que el Editor pueda notificarlos indirectamente a los Suscriptores.

Por ejemplo, podemos definir una clase *AdministradordeEventos* que conserve los mapeos entre eventos y suscriptores (es a la vez una *Fabricación Pura* y una *Indirección* en el sentido de los patrones GRASP). El editor publica un Eevento enviando un mensaje *señaldeEventos* al *AdministradordeEventos*. Cuando se publica, éste encuentra todos los suscriptores a quienes interesa el evento y les envía una notificación, en teoría a través de un mensaje parametrizado, u objeto *respuesta* que dan al hacer la suscripción (figura 22.9). El evento se representa como una cadena simple; lo más probable es que se trate de una instancia de una clase *Evento*. En un sistema de eventos creado en casa, normalmente habrá una sola instancia de la clase *AdministradordeEventos*, a la cual se accede globalmente por medio del patrón Singleton (que se explica más adelante).

El diseño de *AdministradordeEventos* es un ejemplo de conceptos simple e independiente del lenguaje; deben utilizarse las herramientas propias del lenguaje cuando se disponga de ellas. Así, Java especifica un modelo del evento delegación que soporta las metas de publicar-suscribir. Los objetos suscriptor implementan una interfaz *OyentedeEventos* y se suscribe a los eventos al registrarse con los objetos emisor de los eventos (los editores). Los mismos objetos emisor conservan una lista de todos los oyentes registrados y les dan una notificación siempre que ocurre un evento de interés para ellos. El acoplamiento se reduce al definir los suscriptores con una interfaz de Java y no con una clase.

Un diseño clásico de Publicar-Suscribir hace que el objeto editor tenga visibilidad directa de su colección de suscriptores a quienes interesa que se les notifiquen los cambios del editor. Esto tiene la desventaja de impactar el acoplamiento y la implementación de los editores. En cambio, una clase indirecta de *AdministradordeEventos* que cumpla esta función con los suscriptores ofrece la ventaja de reducir al mínimo el acoplamiento y las responsabilidades de los editores aunque puede ocasionar problemas de desempeño, pues es un cuello de botella de todos los eventos.

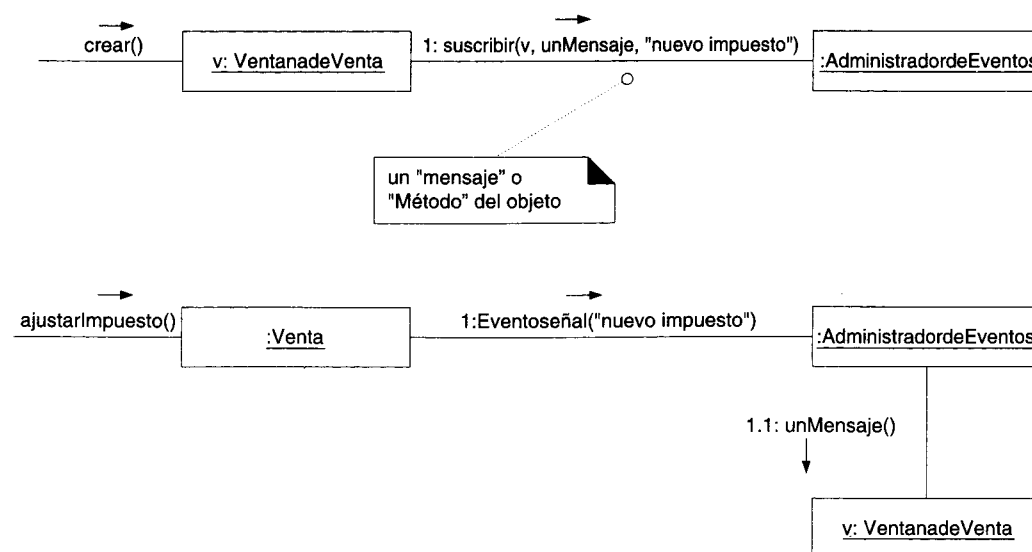


Figura 22.9 Publicar-Suscribir con notificación de los eventos.

22.10.2 Respuestas

El diseño de la figura 22.9 exige transmitir un mensaje parametrizado y un receptor potencial al *AdministradordeEventos*. Podemos encapsular el mensaje y el receptor dentro de una clase de *Respuesta*.¹ Después al *AdministradordeEventos* se le transmite una instancia *Respuesta*, que ejecuta al señalar un evento enviándole a la *Respuesta* un mensaje *ejecutar*.²

El uso de los objetos *Respuesta* tiene la ventaja de ocultar los detalles del receptor y del mensaje, además de que simplifica las responsabilidades del usuario de la *Respuesta*.

22.10.3 Sistemas de notificación de eventos

La arquitectura de Publicar-Suscribir proporciona un mecanismo de propósito general para la notificación de eventos y la comunicación indirecta en un sistema. Se origina así un nuevo método de diseñar los sistemas orientados a objetos: un diseño basado en la notificación de eventos. He aquí sus cualidades distintivas:

¹ Por ejemplo, utilizar el API Reflection de Java o una instancia de clase interna de este lenguaje.

² En Java, el diseño presenta variaciones que no requieren una clase explícita de *Respuesta*. Esto es posible con interfaces, con las clases internas anónimas y también con el modelo de eventos de delegación.

- No se requiere el acoplamiento directo entre emisores y receptores.
- Un evento individual puede transmitirse a cualquier número de suscriptores.
- Puede generalizarse la reacción ante un evento en los objetos *Respuesta*.
- Es relativamente fácil lograr la concurrencia con sólo ejecutar cada *Respuesta* en su propio subproceso múltiple (thread).

Además de las ventajas de un menor acoplamiento, de la transmisión y de una concurrencia simple, algunas veces un sistema basado en la notificación de eventos alcanza mayor eficiencia cuando las largas cadenas de mensajes pueden compactarse en un nivel de indirección.

Finalmente, un diseño de sistemas que se base principalmente en notificar asincrónicamente los eventos y en transmitírselos a los suscriptores va adquiriendo cada vez más la forma de un diseño de estado-máquina. En un capítulo posterior trataremos el tema de la modelación del estado de los objetos.

22.11 Coordinadores de las aplicaciones

El **coordinador de aplicaciones** es una clase que tiene la responsabilidad de mediar entre la capa de interfaz (ventanas, por ejemplo) y la del dominio. Tiene las siguientes responsabilidades fundamentales:

- Mapear la información entre los objetos del dominio y la interfaz. Por ejemplo, consolidar y transformar la información proveniente de uno o varios objetos del dominio.
- Responder a los eventos procedentes de la interfaz.
- Abrir ventanas que muestren la información proveniente de los objetos del dominio.
- Administrar las transacciones; por ejemplo, realizar las operaciones "commit" y "rollback".

Si la arquitectura tiene por objeto soportar vistas o ventanas múltiples que despliegan la misma información, los coordinadores de aplicación brindarán además soporte a las siguientes responsabilidades de vistas múltiples:

- Soportar la capacidad de que varias ventanas ("ventanas dependientes") muestren simultáneamente la información procedente de una instancia coordinador de aplicaciones.
- Notificarles a las ventanas dependientes cuando cambie la información y cuando las ventanas necesitan refrescarse (repintarse) con nueva información.

Algunos esquemas de aplicaciones incluyen el soporte de alguna modalidad de coordinador. Por ejemplo, en la aplicación MFC los coordinadores son *Documentos* en la arquitectura Documento-Vista; son subclases de la clase *CDocument*.¹

¹ En Smalltalk de VisualWorks, un *ApplicationModel* desempeña el papel de un coordinador de aplicaciones. En Visual Smalltalk, esto lo hace un *ViewManager* o *ApplicationCoordinator*.

En la figura 22.10 encontrará el lector un ejemplo concreto de la relación existente entre varios componentes y la arquitectura Documento-Vista de MFC. El atributo visibilidad se indica gráficamente en los enlaces del diagrama de colaboración. Adviértase que —en apoyo del patrón de Separación Modelo-Vista—, los objetos de dominio *TPDV* y *Venta* no tienen visibilidad de *DocumentodeVenta* ni *VistadeVenta*.

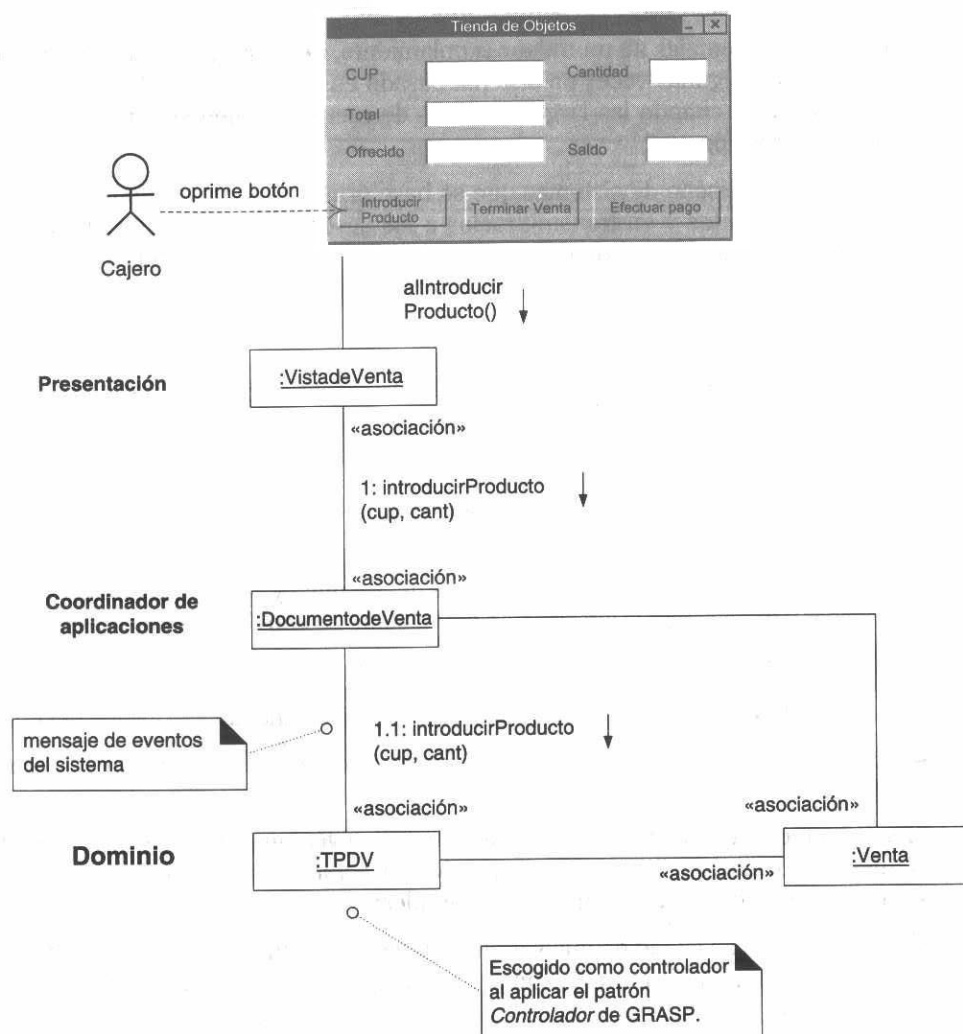


Figura 22.10 Colaboración y visibilidad entre objetos en varias capas de la arquitectura de Documento-Vista de Clases de la Microsoft Foundation (MFC).

El *DocumentodeVenta* (el coordinador de aplicaciones) tiene visibilidad de *TPDV* como el controlador escogido (en el sentido de los patrones GRASP) para los mensajes de operación del sistema y también tiene visibilidad de la *Venta* para la cual se va a mostrar información en la ventana *VistadeVenta*. El *DocumentodeVenta* estableció el atributo de la visibilidad de la *Venta* al solicitársela a *TPDV*, el creador de ella.

Considere el siguiente ejemplo de flujo de control:

1. El cajero oprime un botón de la ventana.
2. El evento se envía como mensaje *allIntroducirProducto* a la instancia *VistadeVenta*, que representa la ventana exhibida.
3. La instancia *VistadeVenta* envía a *DocumentodeVenta* el evento como el mensaje *introducirProducto*.

DocumentodeVenta envía el mensaje *introducirProducto* (el mensaje de eventos del sistema) al controlador *TPDV* en la capa del dominio.

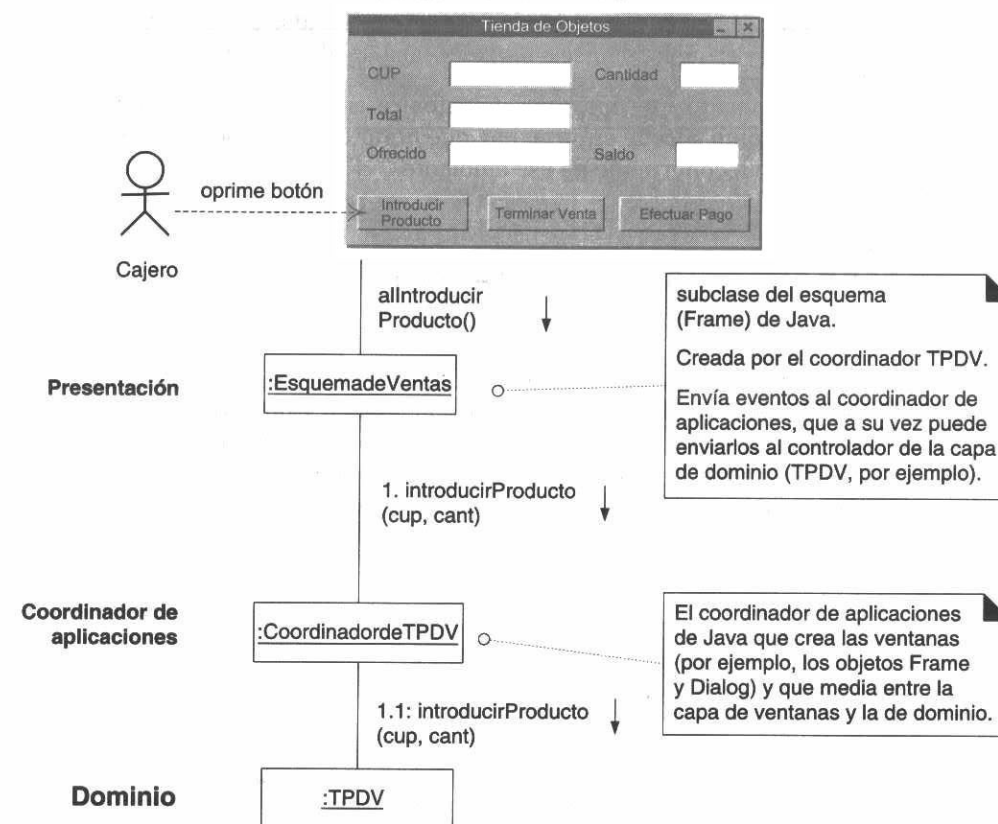


Figura 22.11 Agregado de un coordinador de aplicaciones a una aplicación de Java.

22.11.1 Los coordinadores de aplicaciones y los objetos ventana

Cuando los coordinadores de aplicaciones no cuentan con un soporte explícito de esquema, es común que la clase ventana o vista asuman algunas de las responsabilidades de ellos. Por ejemplo, JDK de Java no ofrece una coordinación in-

dividual de aplicaciones a partir de la presentación. Si se utiliza un applet de Java, casi siempre cumple con las responsabilidades de coordinar la presentación y la aplicación.

22.11.2 Coordinadores de las aplicaciones diseñados en casa

Aun cuando los coordinadores de aplicaciones carezcan del soporte de un esquema, podemos incorporarlos para llevar a cabo la administración de transacciones, notificar muchas ventanas dependientes y mapear la información.

Por ejemplo, podemos diseñar una aplicación (no un applet) de modo que la clase *CoordinadordeTPDV*, que representa al coordinador de aplicaciones, medie entre ventanas (por ejemplo, un *Esquema*) y la capa del dominio (figura 22.11).

Si no se incluye el soporte arquitectónico anterior para los coordinadores de aplicaciones, como sucede en Java, hay que obrar con cautela al incorporarlos, pues son un elemento incongruente en ese ambiente.

22.12 Almacenamiento y persistencia

Si se emplea una base de datos de objetos para guardar los objetos capa de dominio, los diseñadores no tendrán que hacer mucho esfuerzo para interactuar con ella: el proveedor les proporciona una interfaz. Pero si utiliza una base de otro tipo (por ejemplo, una base de datos relacional), se requerirá un subsistema (figura 22.12) que genere un mapeo entre los objetos y los renglones de las tablas. En el capítulo 38 veremos cómo se diseña.

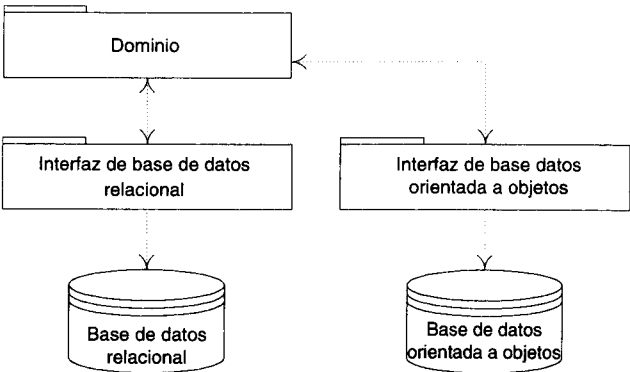


Figura 22.12 Subsistemas de persistencia.

22.13 Modelos muestra

Los diagramas de paquetes arquitectónicos son una parte del Modelo de Arquitectura: el de capas y subsistemas del sistema y del despliegue de los componentes para los procesadores.

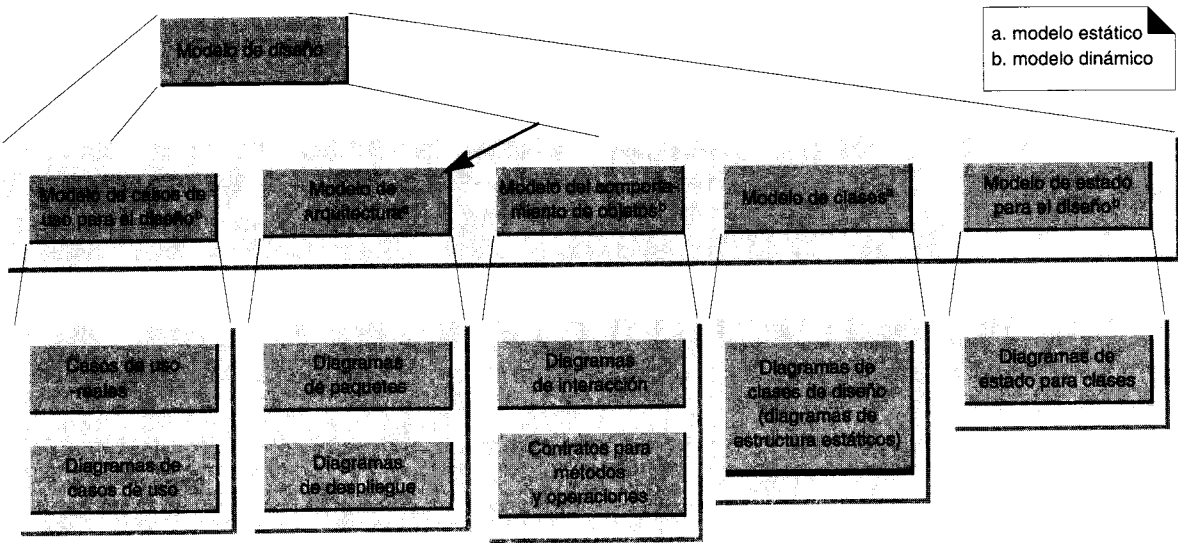


Figura 22.13 El modelo de diseño.

PARTE V **FASE DE
CONSTRUCCIÓN (1)**

MAPEO DE LOS DISEÑOS PARA CODIFICACIÓN

Objetivos

- Mapear los artefactos del diseño para realizar la codificación en un lenguaje orientado a objetos.

23.1 Introducción

Una vez concluidos los diagramas de clases del diseño y destinados al ciclo de desarrollo actual en la aplicación punto de venta, dispondremos de suficientes detalles para generar un código que utilizaremos en la capa del dominio de los objetos.

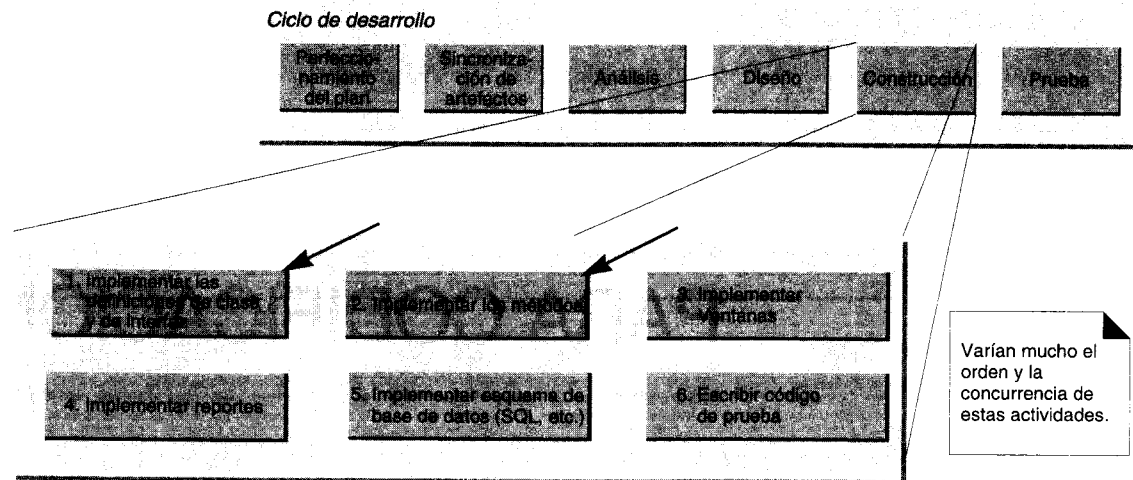
Los artefactos del UML creados en la fase de diseño —los diagramas de colaboración y los de clases del diseño— servirán de entrada en el proceso de generación del código.

23.2 La programación y el proceso de desarrollo

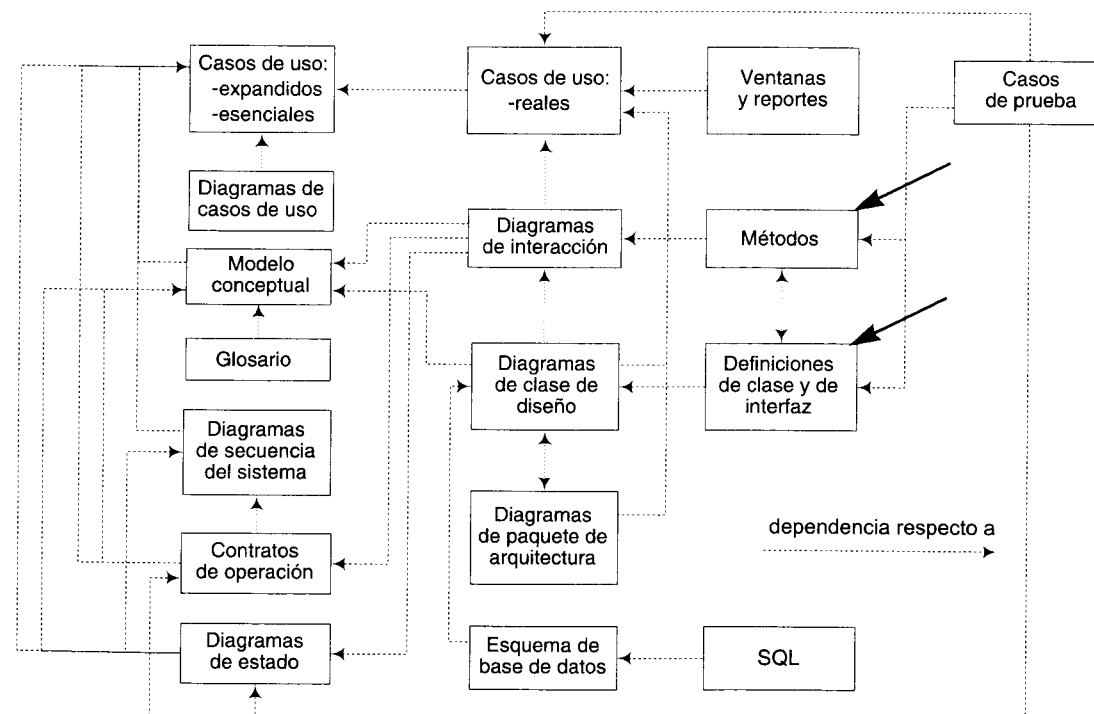
Si se quiere reducir el riesgo y aumentar la probabilidad de conseguir una aplicación adecuada, el desarrollo debería basarse en un suficiente modelado del análisis y diseño antes de iniciar la codificación.¹ Ello no significa que durante la programación no tengan cabida los prototipos ni el diseño: las modernas herramientas del desarrollo ofrecen un excelente ambiente para examinar rápidamente métodos alternos, y normalmente vale la pena dedicar poco o mucho tiempo al diseño por la codificación.

¹ “Software” es una palabra que no hace mucho honor a su significado (“blando”). La modificación y el examen de alternativas son más difíciles y costosos durante la fase de programación que durante la de análisis y diseño.

Ciclo de desarrollo



Actividades de la fase de construcción dentro de un ciclo de desarrollo.



Dependencias de los artefactos durante la fase de construcción.

Pero la parte esencial de la aplicación —por ejemplo, el modelo conceptual básico, las capas arquitectónicas, las principales asignaciones de responsabilidades y las interacciones más importantes de los objetos— se determina más satisfactoriamente en una investigación formal y en el proceso de diseño que “apresurándose a codificar”. Esto último origina sistemas que son más difíciles de entender, ampliar y de darles mantenimiento, sin que brinden soporte a un proceso susceptible de repetirse eficazmente.

Dicho lo anterior, a menudo lo más conveniente es omitir la fase de diseño para realizar un poco de programación exploratoria a fin de descubrir un diseño funcional y luego regresar a la fase formal de diseño.

La creación de código en un lenguaje orientado a objetos —Java y Smalltalk, entre otros— no forma parte del análisis ni del diseño orientados a objetos; es meta final en sí misma. Los artefactos producidos en la fase de diseño suministran suficiente información necesaria para generar el código y, como veremos luego, se trata de un proceso de traducción relativamente simple.

Una ventaja del análisis, del diseño y de la programación orientados a objetos —cuando se emplean junto con un proceso de desarrollo como el que hemos recomendado— es que ofrece una guía completa de principio a fin para realizar la codificación a partir de los requerimientos. Los artefactos que se introducen en otros posteriores en una forma rastreable y útil culminarán finalmente en una aplicación funcional. Con ello no queremos decir que el camino sea fácil ni que podamos seguirlo mecánicamente, pues existen demasiadas variables. Pero la guía constituye un punto de partida para experimentar e intercambiar opiniones.

23.2.1 Creatividad y cambio durante la fase de construcción

La toma de decisiones y el trabajo creativo se realizaron en gran medida durante las fases de análisis y de diseño. En la siguiente exposición veremos que la generación del código —en nuestro ejemplo— es un proceso de traducción bastante mecánico.

No obstante, en general la fase de programación no es un paso fácil de la generación del código, todo lo contrario. En realidad los resultados obtenidos durante el diseño son un primer paso incompleto; en la programación y en las pruebas se efectuará multitud de cambios y se descubrirán y resolverán problemas detallados.

Los artefactos del diseño, cuando están bien hechos, producen un núcleo flexible que crece con elegancia y fuerza para atender los nuevos problemas que surjan en la programación.

En consecuencia, espere y planee para afrontar el cambio y la desviación respecto al diseño durante la fase de construcción y de pruebas.

23.2.2 Cambios de código y el proceso iterativo

Una de las ventajas del proceso de desarrollo iterativo e incremental es la posibilidad de introducir los resultados de un ciclo anterior al iniciar el siguiente. Así pues, los resultados del análisis y diseño subsecuentes se perfeccionan sin cesar y aprovechan